
Predictive Coding from Scratch: A Reproducible Study of Local Learning versus Backpropagation, with a Fused CUDA Settling Kernel

An independent, self-directed research project (master's-thesis scope)

by
Sven Weidenmann

2026

Source code, experiments, and figures:
<https://github.com/AlsoTheZv3n/PCN-self-improvement-research>

Statement on Independence and Tools

This is an independent, self-directed research project, written outside any academic institution and not submitted for any degree. It is released publicly as a portfolio and learning artifact.

The research questions, design decisions, interpretation of results, and the honest framing of the (largely null) findings are my own. The implementation, experiments, and this document were produced with substantial assistance from an AI coding assistant (Anthropic's Claude) acting under my direction; I take responsibility for the content. All quantitative results are reproducible from the released code and artifacts, and all external work (theory, prior implementations, datasets) is cited.

Abstract

Predictive Coding (PC) is a biologically motivated, local-learning alternative to backpropagation (BP) whose reported advantages are inconsistent across the literature. This thesis builds a from-scratch State-Optimization PC network on MNIST in pure PyTorch—no autograd in the learning rule; the hand-derived settling and weight gradients are verified against autograd—and uses it for a deliberately sober, reproducible study with three contributions. **(i) Systems:** among open PC libraries, none ships a custom inference kernel; we contribute, to our knowledge, the first hand-written *fused CUDA settling kernel* for PC, show that PC inference at MNIST scale is launch-overhead-bound, and obtain $3.2\times$ at batch 64, generalizing the kernel to arbitrary depth and using it as the engine of one of our own studies ($1.45\times$ end-to-end, identical results). **(ii) Methodology:** a confound-controlled fair PC-vs-BP comparison (matched architecture, initialization, data and loss; per-method learning-rate tuning; bootstrap confidence intervals over multiple seeds) finds $PC \approx BP$ across accuracy, robustness, sample efficiency and continual learning, including an architecture-faithful replication of Song et al.’s exact alternating Split-FashionMNIST protocol where no PC-vs-BP difference exceeds one standard deviation at any training budget. We isolate three confounds (loss function, the stability–plasticity trade-off, and an untuned baseline) that manufacture spurious “PC advantages”, and describe an adversarial verification protocol that caught a literature-framing error before it entered this document. **(iii) Capability:** a generatively trained PC recovers the “one mechanism, many tasks” property—classification, generation, inpainting, and anomaly detection from a single settling rule. We additionally implement incremental PC and an EWC baseline. All code, tests, figures, and experiment artifacts are released publicly.

Zusammenfassung

Predictive Coding (PC) ist eine biologisch motivierte, lokal lernende Alternative zur Backpropagation (BP), deren berichtete Vorteile in der Literatur uneinheitlich sind. Diese Arbeit implementiert ein From-Scratch-PC-Netz (State-Optimization-Formulierung) auf MNIST in reinem PyTorch—ohne Autograd in der Lernregel; die handabgeleiteten Gradienten sind gegen Autograd verifiziert—und nutzt es für eine bewusst nüchterne, reproduzierbare Studie mit drei Beiträgen. **(i) Systems:** der erste handgeschriebene, fusionierte CUDA-Settling-Kernel für PC (nach unserem Kenntnisstand); PC-Inferenz ist auf MNIST-Skala launch-overhead-gebunden, der Kernel erreicht $3,2\times$ bei Batch 64, ist auf beliebige Tiefe verallgemeinert und treibt eine der Studien ($1,45\times$, identische Ergebnisse). **(ii) Methodik:** ein confound-kontrollierter, fairer PC-vs-BP-Vergleich findet $PC \approx BP$, inklusive einer architektur-treuen Replikation von Songs exaktem alternierenden Protokoll (kein Unterschied $> 1\sigma$ bei keinem Budget). Drei Confounds (Loss-Funktion, Stabilität-Plastizität, ungetunte Baseline) erzeugen scheinbare PC-Vorteile; eine adversariale Verifikation fing einen Framing-Fehler vor Eingang in diese Arbeit. **(iii) Fähigkeit:** ein generativ trainiertes PC liefert die “eine Maschinerie, viele Aufgaben”-Eigenschaft. Zusätzlich sind inkrementelles PC und eine EWC-Baseline implementiert. Aller Code, Tests, Figuren und Artefakte sind veröffentlicht.

Contents

Abstract	v
1 Introduction	1
1.1 Motivation	2
1.2 Problem Statement and Research Questions	2
1.3 Contributions	3
1.4 Scope and Non-Goals	4
1.5 Outline of the Thesis	4
2 Background	5
2.1 Notation and the MLP Baseline	5
2.1.1 The feedforward MLP	6
2.1.2 Backpropagation as the reference	6
2.2 Predictive Coding: The Energy Formulation	6
2.2.1 Prediction errors and the energy	7
2.2.2 Isotropic precision as the modern default	7
2.3 State-Optimization PC: Settling and the Local Update	8
2.3.1 Clamping conventions	8
2.3.2 Feedforward initialization	9
2.3.3 The settling dynamics	9
2.3.4 The signal-decay caveat	10
2.3.5 The local Hebbian weight update	10
2.4 Relation to Backpropagation	11
2.4.1 The fixed-prediction equivalence	11
2.4.2 Prospective configuration is the equilibrium, not a separate algorithm	12
2.5 When Does PC Equal BP?	12
2.5.1 The provable equivalence regimes are narrow	12
2.5.2 The role of the settling budget	13

2.6	Generative versus Discriminative PC	13
2.6.1	Discriminative orientation	13
2.6.2	Generative orientation	14
3	Related Work	15
3.1	Predictive Coding Implementations and Libraries	16
3.1.1	The two implementation idioms: JAX/JIT and PyTorch-manual	16
3.1.2	The audit: eleven libraries, zero settling kernels	17
3.2	Equilibrium Propagation: The Closest Algorithmic Relative	18
3.2.1	The canonical and recent EP implementations	18
3.2.2	The single near-miss, and why it does not refute the claim . .	19
3.2.3	A scope caveat: analog and neuromorphic acceleration	19
3.3	Systems for Iterative / Fixed-Point Inference	20
3.3.1	The persistent, single-launch kernel as prior art	20
3.3.2	Deep Equilibrium models and the conceptual neighbours . . .	21
3.3.3	The framework tax: a generic observation, not a contribution .	21
3.4	Continual Learning	21
3.4.1	Catastrophic forgetting and the stability–plasticity trade-off .	22
3.4.2	Measuring forgetting: the GEM metrics	22
3.4.3	Regularisation baselines: Elastic Weight Consolidation	23
3.5	Positioning of This Work	23
4	Method and Implementation	25
4.1	From-Scratch PCN Architecture and Interface	26
4.1.1	The model object	26
4.1.2	Feedforward initialisation, settling, and the local update . . .	26
4.1.3	The convergence criterion	27
4.1.4	The <code>train_and_eval(config)</code> contract	28
4.2	Correctness: Hand-Derived Gradients vs. Autograd	28
4.3	The Fused CUDA Settling Kernel	29
4.3.1	Motivation: predictive-coding inference is launch-overhead-bound	29
4.3.2	Novelty and honest scope	30
4.3.3	The fused design and its three versions	30
4.3.4	The general-depth variant	31
4.3.5	Correctness before speed	32
4.4	The Fair-Comparison Protocol	32
4.4.1	Identical architecture, initialisation, and forward function . . .	32

4.4.2	Matched loss — the dominant confound	33
4.4.3	Joint learning-rate tuning and the stability frontier	33
4.4.4	Statistics: validation selection and bootstrap intervals	33
4.5	Continual-Learning Setups	34
4.5.1	Metrics: the R-matrix, ACC, BWT, and learn-accuracy	34
4.5.2	Domain-incremental: Permuted-MNIST	34
4.5.3	Class-incremental: Split-MNIST and Split-FashionMNIST	35
4.5.4	The Song-exact alternating protocol	35
4.6	Generative PC	36
4.6.1	Orientation flip and generative training	36
4.6.2	Masked and input-free settling	36
4.6.3	The output-dimension stability scaling	36
4.7	Optional Extensions: Incremental PC and EWC	37
4.7.1	Incremental PC	37
4.7.2	Elastic Weight Consolidation	37
4.8	Reproducibility and Tooling	38
5	Experiments and Results	41
5.1	Experimental Setup	41
5.1.1	Hardware, software, and reproducibility	41
5.1.2	Datasets and architecture	42
5.1.3	Metrics	42
5.1.4	Statistics	43
5.2	The Fused CUDA Settling Kernel	43
5.2.1	Correctness comes first	43
5.2.2	PC inference is launch-overhead-bound	44
5.2.3	Speedup and the architectural crossover	45
5.2.4	Generality and in-study use	46
5.2.5	Novelty scope	47
5.3	Fair PC versus BP	47
5.3.1	A methodological prerequisite: tune η_x and η_w jointly	47
5.3.2	The confounded first results and their correction	48
5.3.3	Bulk accuracy and noise robustness	48
5.3.4	Continual learning: domain-incremental	49
5.3.5	Continual learning: class-incremental (Song’s regime)	50
5.3.6	Synthesis: three confounds, one null	50
5.4	Architecture-Faithful Replication of Song’s Exact Protocol	51
5.4.1	What was built	51

5.4.2	The framing correction (an adversarial three-lens review) . . .	51
5.4.3	Budget as a controlled axis	52
5.4.4	Results	52
5.4.5	Why the kernel matters here	53
5.5	Generative Predictive Coding	53
5.5.1	A discriminative PC cannot generate	54
5.5.2	A generative PC delivers the full capability set	54
5.6	Optional Extensions: iPC and EWC	55
5.6.1	Incremental PC (iPC)	55
5.6.2	EWC as a continual-learning positive control	55
5.7	Autonomous Hyperparameter Search	56
5.8	Depth, Signal Decay, and a Local Precision Mitigation	56
5.8.1	Reproducing the signal decay	56
5.8.2	A local precision schedule mitigates it — but is a known technique	57
6	Discussion	59
6.1	Are Predictive Coding’s Advantages Real?	59
6.1.1	What “parity” does and does not mean	60
6.1.2	Why the literature disagrees	60
6.2	The Confound Checklist as a Transferable Contribution	61
6.2.1	The three confounds	62
6.2.2	Why the checklist transfers	62
6.3	The Systems Insight: Launch Overhead and the GEMM Trade-off . .	63
6.3.1	PC inference is launch-overhead-bound, and that is a framework tax	63
6.3.2	The architectural trade-off bounds the win	64
6.3.3	Why this matters for PC specifically	64
6.4	What a Generative PC Buys	65
6.4.1	The orientation, not the mechanism, is decisive	65
6.4.2	Honest reading of the capability	65
6.5	Methodology: Adversarial Verification	66
6.5.1	The three-lens review and the error it caught	66
6.5.2	Why this is a reusable safeguard, not a one-off	66
6.6	Threats to Validity	67
6.6.1	Internal validity	67
6.6.2	External validity	68
6.6.3	Construct validity	68

7	Limitations and Future Work	69
7.1	Limitations	69
7.1.1	Scope of the scientific findings	70
7.1.2	Limitations of the systems contribution	71
7.1.3	Limitations of the capability demonstrations	73
7.1.4	Limitations of experimental reach	74
7.2	Future Work	75
7.2.1	Extending the systems contribution	75
7.2.2	Extending the scientific contribution	76
7.2.3	Extending the capability demonstrations	77
8	Conclusion	79
A	Appendix	81
A.1	Exact Settling and Weight-Update Derivation	81
A.2	Kernel Implementation Details	82
A.3	Additional Results	83
A.4	Reproducibility	83

List of Figures

- 5.1 CUDA kernel launches per full settle versus the number of settling steps T (batch 64, RTX 3080 Ti). The PyTorch SO backend issues 31 launches per step ($620 \rightarrow 1240 \rightarrow 2480$ for $T = 20, 40, 80$); the fused kernel issues exactly one, independent of T . This is the mechanistic confirmation that PC inference at this scale is launch-overhead-bound rather than compute-bound. 44
- 5.2 Fused CUDA settling-kernel speedup over the PyTorch backend versus batch size (RTX 3080 Ti, MLP [784, 256, 256, 10], settling steps $T = 20$ and 40). The kernel wins at small batch (launch-overhead bound, $\sim 3.2\times$ at $B = 64$, $\sim 2.0\times$ at $B = 256$), reaches break-even around $B \approx 1024$, and is outperformed by cuBLAS at $B \geq 2048$, where the workload becomes compute-bound. The small/medium-batch regime is the one relevant to online and continual PC training. 46
- 5.3 Matched-loss PC versus BP on MNIST (3 seeds, 68% bootstrap CIs). *Left*: bulk test accuracy — PC(MSE) and BP(MSE) are indistinguishable (CIs overlap); BP(CE) leads by the +5.9 pp loss effect, not by the learning rule. *Right*: test accuracy versus additive input-noise σ — BP(MSE) is modestly more robust than PC(MSE), but the large naive gap is again mostly the loss function. 49
- 5.4 Song-exact alternating Split-FashionMNIST: mean-both (Song’s metric) and worse-task min-both (interference tell) accuracy versus training budget ($n = 10, \pm 1\sigma$). No PC-vs-BP(MSE) gap exceeds 1σ at any budget. PC learns slower early and pulls marginally ahead by convergence, but the gap stays inside the (large) seed spread; raising the seed count from five to ten did not make it significant. BP(CE) leads throughout — the same loss confound as Section 5.3. 53

- 5.5 Left: test accuracy versus depth (width 64, MNIST). Standard SO-PC matches BP at one–two hidden layers but collapses as depth grows, while the matched BP baseline holds; a local precision schedule $\Pi_k = \gamma^{(n-k)}$ recovers much of the gap. Right: per-layer learning signal $\|\Delta W_i\|$ at the deepest network — it decays exponentially toward the input under standard SO-PC (output/input ratio $\sim 669\times$) and is flattened $\sim 10\text{--}30\times$ by the precision schedule. 57

List of Tables

5.1	Deconfounded bulk comparison on MNIST (10k examples, matched MSE loss unless noted; mean over 3 seeds with 68% bootstrap CI). The fair PC-vs-BP(MSE) gap is +0.4 pp with overlapping CIs; the loss effect $\text{BP(CE)} - \text{BP(MSE)}$ is +5.9 pp.	48
5.2	Permuted-MNIST domain-IL (5 tasks, matched loss; mean over 3 seeds, 68% CI on retained task-0). Against fair BP(MSE), BP forgets <i>less</i> and learns each task better; the earlier “PC forgets less” was an artifact of comparing to BP(CE).	49
5.3	Class-IL parity (mean over 3 seeds, 68% CI on retained task-0). Against a correctly tuned, <i>learning</i> BP(MSE) baseline, PC shows no advantage in either split.	50
5.4	Song-exact alternating Split-FashionMNIST, ten seeds. Mean-both and worse-task min-both accuracy (%) with $\pm 1\sigma$ bootstrap intervals; $\Delta_{\min}$ is PC – BP(MSE). At <i>no</i> budget does the PC-vs-BP(MSE) gap exceed 1σ (the seed spread σ is ~ 4 –11 pp on each arm). The pattern is directional only.	52
5.5	The same settling rule, two training orientations. Generative PC recovers the capabilities a discriminative PC cannot provide.	54
5.6	Test accuracy (%) versus depth (width 64, MNIST, 3 seeds). SO-PC collapses with depth while matched BP holds; a local precision schedule recovers much of the gap at moderate depth, with the optimal γ falling as the γ^{depth} gain compounds.	57

Chapter 1

Introduction

Backpropagation is the engine of modern deep learning, yet it is widely regarded as biologically implausible. Training a network by the chain rule requires a global backward pass that transports the transpose of each weight matrix back through the network (the *weight-transport problem*), that locks every layer’s update until the full forward and backward sweeps complete, and that carries an error signal which is mechanistically distinct from the activity used in the forward pass. Cortical circuits exhibit none of these properties: they learn with local signals, in place, and apparently without a separate backward channel. This mismatch has motivated a sustained search for *local* learning rules that could, in principle, be implemented by biological or neuromorphic substrates while still solving the credit-assignment problem that backpropagation solves globally.

Predictive Coding (PC) is among the most developed of these alternatives. Originating as a theory of cortical function (Rao and Ballard, 1999) and later cast in the language of the free-energy principle (Bogacz, 2017), PC frames perception and learning as the minimisation of a layer-wise, precision-weighted prediction-error energy. Inference is an iterative *settling* process that relaxes the network’s internal states toward a configuration of low prediction error, after which a purely local Hebbian rule updates the weights. PC is attractive precisely because it is local, and it has accumulated a set of appealing claims: that it approximates backpropagation (Whittington and Bogacz, 2017); that it can learn from fewer examples; that it forgets less in continual-learning settings; and that the very same machinery doubles as a generative model. If even some of these held robustly, PC would be more than a biological curiosity.

The difficulty is that the evidence is inconsistent. Reported advantages appear in some studies and regimes and vanish in others, and comparisons across the literature are rarely controlled for the many degrees of freedom that separate a PC network

from its backpropagation baseline (Zahid et al., 2023). At the same time, PC’s iterative inference makes it markedly more expensive to run than a feedforward pass, and the engineering question of *where that cost actually goes* on modern hardware has received little attention. This thesis takes a deliberately sober stance toward both issues. It does not attempt a breakthrough; it builds a clean, from-scratch PC implementation and uses it to ask careful, falsifiable questions, reporting the answers—several of them null—honestly.

1.1 Motivation

The motivation for this work is threefold. *Scientifically*, the inconsistency in the PC-versus-backpropagation literature is itself a problem worth addressing: if the reported advantages are real, a controlled study should recover them, and if they are artefacts, a controlled study should expose the artefacts. Either outcome is informative. *From a systems perspective*, PC’s settling loop is an unusual workload—many small, sequential, launch-bound operations rather than a few large matrix multiplications—and characterising its cost is both practically useful and a natural bridge to the wider literature on accelerating iterative, fixed-point computations. *Pedagogically*, implementing PC from first principles, with no reliance on automatic differentiation for the learning rule, forces every equation to be understood and verified, which is the surest route to separating what PC genuinely does from what a high-level framework does on its behalf.

A recurring theme throughout the thesis is that fair comparison is hard, and that most apparent PC advantages dissolve once the comparison is made fair. Rather than treat this as a disappointment, we treat methodological honesty—naming the confounds, controlling them, reporting confidence intervals and seed counts, and declining to call a marginal effect significant—as the central contribution.

1.2 Problem Statement and Research Questions

We organise the work around three research questions, each of which maps to one of the contributions below.

RQ1 (Science). Under a comparison that matches architecture, initialisation, data, and loss function, tunes each method’s learning rate fairly, and reports uncertainty over multiple seeds, does Predictive Coding actually outperform backpropagation at MNIST scale—on accuracy, robustness, sample efficiency,

or continual learning—and if not, what explains the advantages reported elsewhere?

RQ2 (Systems). What dominates the cost of PC inference on a GPU, and can that cost be reduced by a dedicated implementation of the settling loop?

RQ3 (Capability). Can a single from-scratch PC network act as one mechanism for many tasks—classification, generation, inpainting, and anomaly detection—and under what conditions does this property actually emerge?

These questions are intentionally answerable at the scale we can afford to study rigorously. They trade breadth for the ability to give each answer with controlled experiments and honest uncertainty, rather than gesturing at trends.

1.3 Contributions

The thesis makes three contributions, aligned with the research questions.

(A) A fused CUDA settling kernel and a characterisation of PC’s inference cost.

We contribute, to our knowledge, the first hand-written fused CUDA kernel for the PC settling loop. The persistent, states-resident, single-launch *technique* is established prior art in other domains; our contribution is its transfer to PC together with the measurement that PC inference at MNIST scale is launch-overhead-bound. The kernel yields a $3.2\times$ speedup at small batch, generalises to networks of arbitrary depth, and is fast enough to serve as the actual backend of one of our own studies (Chapters 4 and 5).

(B) A confound-controlled fair-comparison protocol and a parity finding.

We design a PC-versus-backpropagation protocol that holds everything but the learning rule fixed, and find parity across every axis we test—including an architecture-faithful replication of an exact protocol from the literature. We isolate three confounds that manufacture spurious “PC advantages” in naive comparisons, and we describe an adversarial verification procedure that caught a literature-framing error in our own draft before it reached this document (Chapters 5 and 6).

(C) A working generative PC, plus two implemented extensions. We

show that the “one mechanism, many tasks” property is real only for a *generatively* oriented PC, and demonstrate classification, generation, inpainting, and anomaly detection from a single settling rule. We additionally implement

incremental PC and an Elastic Weight Consolidation baseline (Chapters 4 and 5).

All code, the test suite, the figures, and the raw experiment artifacts are released publicly so that every number in this thesis can be reproduced.

1.4 Scope and Non-Goals

This is a study at MNIST and FashionMNIST scale, with shallow multilayer perceptrons (two to four hidden layers) and the vanilla State-Optimization formulation of PC. It is explicitly *not* a claim about Transformer-scale models, nor a state-of-the-art result, nor a refutation of any prior work in that work’s own regime. The honest framing of the project is that a from-scratch MNIST PC network is primarily an exercise in *understanding and clean reproduction*, and that whatever novelty it carries comes from a small number of focused, defensible angles—a systems measurement, a methodological checklist, and a capability distinction—rather than from a paradigm shift. Chapter 7 states the boundaries of every claim in full.

1.5 Outline of the Thesis

Chapter 2 develops the State-Optimization formulation of PC: the prediction-error energy, the settling dynamics and the local weight update, the precise sense in which PC relates to backpropagation, and the orientation distinction between discriminative and generative PC. Chapter 3 surveys the prior art—PC libraries, Equilibrium Propagation as the closest algorithmic relative, systems work on fixed-point inference, and the continual-learning literature—and positions the thesis’s claims, in particular grounding the systems-novelty claim in a direct source audit. Chapter 4 describes what was built: the from-scratch network and its single training interface, the gradient checks against autograd, the fused CUDA kernel, the fair-comparison protocol, the continual-learning setups including the exact replication, the generative model, and the incremental-PC and EWC extensions. Chapter 5 reports the measurements—the kernel benchmark and launch-count analysis, the fair PC-versus-BP study and the three confounds, the budget-swept exact replication, the generative results, and the extensions. Chapter 6 interprets the findings, develops the confound checklist and the systems insight as transferable contributions, and reflects on the adversarial-verification methodology. Chapter 7 states the limitations and the concrete future work that would extend the claims, and Chapter 8 concludes.

Chapter 2

Background

This chapter develops the theoretical machinery that the rest of the thesis rests upon. It is deliberately self-contained and pedagogical: we begin from the familiar feedforward multilayer perceptron (MLP) trained by backpropagation, introduce the predictive-coding (PC) energy that reinterprets a network as an inference problem, derive the State-Optimization (SO) settling dynamics and the local Hebbian weight rule in full, and only then turn to the precise relationship between PC and backpropagation. We restrict ourselves throughout to *theory*. Our concrete implementation choices, the libraries that preceded us, and our empirical results are deferred to Chapters 3 to 5 respectively. The aim here is that a reader who has never seen predictive coding should, by the end of the chapter, be able to read the settling and update equations and understand exactly what each term does and why.

2.1 Notation and the MLP Baseline

We fix notation now and use it consistently for the remainder of the thesis. Consider a network of $n + 1$ layers indexed $0, 1, \dots, n$. Layer 0 holds the input and layer n holds the output; the layers in between are hidden. We write the *state* (the vector of unit activities) of layer k as s_k , the weight matrix mapping layer i to layer $i + 1$ as W_i , and the corresponding bias as b_i . The element-wise activation function is ϕ (we use $\tanh$ as the default throughout, with the sigmoid reserved for one faithfulness replication), and ϕ' denotes its derivative. Layer sizes for our running MNIST example are $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$: a 784-dimensional pixel vector, two hidden layers of width 256, and a 10-dimensional output read as a one-hot class label (LeCun et al., 1998).

2.1.1 The feedforward MLP

The ordinary feedforward MLP computes its activities by a single forward sweep. Writing the pre-activation of layer $i + 1$ as

$$pred_{i+1} = W_i \phi(s_i) + b_i, \quad (2.1)$$

the MLP simply *assigns* $s_{i+1} = pred_{i+1}$ for every layer in turn, starting from the clamped input $s_0 = x$ and ending at the output s_n . We deliberately call the right-hand side of eq. (2.1) a *prediction*: it is the higher layer’s best guess, given s_i , of what the activity of layer $i + 1$ ought to be. In a plain MLP that guess is taken as gospel and copied into s_{i+1} . The central conceptual move of predictive coding, developed below, is to stop treating the prediction as definitive and instead let the actual activity s_{i+1} disagree with it; the discrepancy is precisely what drives learning.

2.1.2 Backpropagation as the reference

Given a differentiable scalar loss $\mathcal{L}(s_n, y)$ comparing the output to a target y , backpropagation (BP) computes the gradient of $\mathcal{L}$ with respect to every weight by the chain rule, propagating an error signal from the output back toward the input. Two properties of this procedure matter for what follows. First, the weight update for layer i depends on a quantity transported from *all* downstream layers: the backward pass threads the transposed weight matrices $W_{i+1}^\top, W_{i+2}^\top, \dots$ in sequence, so the signal reaching layer i has been routed through the entire upper part of the network. This non-locality is the “weight-transport problem” often cited as a reason BP is biologically implausible. Second, BP changes the weights *first*; any change in the network’s activities is a downstream consequence of the next forward pass, not part of the learning step itself. We will see that predictive coding inverts both of these properties: its weight update is strictly local, and it changes activities *before* it touches the weights. Throughout the thesis, a matched-size MLP trained by BP under PyTorch autograd (Paszke et al., 2019) serves as the reference against which PC is measured; the discipline required to make that comparison fair is the subject of Section 4.4.

2.2 Predictive Coding: The Energy Formulation

Predictive coding originates in computational neuroscience as a theory of cortical processing (Rao and Ballard, 1999): higher cortical areas send *predictions* of activity down to lower areas, and only the residual *prediction error*—the part of the lower

activity that the prediction failed to explain—is sent back up. Learning and perception both reduce to making those prediction errors small. Bogacz (2017) recast this picture in the language of the variational free-energy framework and gave the tutorial derivation we follow here; Millidge et al. (2021) survey the multi-layer formulation in detail. The result is a network that is defined not by a forward function but by a single scalar *energy*.

2.2.1 Prediction errors and the energy

For each layer $k \in \{1, \dots, n\}$ we define the prediction error as the difference between the layer’s actual state and the top-down prediction of it from the layer below,

$$\varepsilon_k = s_k - \text{pred}_k = s_k - (W_{k-1} \phi(s_{k-1}) + b_{k-1}). \quad (2.2)$$

The network’s *energy* is the precision-weighted sum of squared prediction errors. In full generality each layer carries a precision matrix Π_k (the inverse covariance of the assumed Gaussian noise on that layer), giving

$$E = \frac{1}{2} \sum_{k=1}^n \varepsilon_k^\top \Pi_k \varepsilon_k. \quad (2.3)$$

This is exactly the Gaussian form of the variational free energy of the free-energy principle, wired layer-by-layer into the network (Bogacz, 2017; Millidge et al., 2021). It is worth stressing what eq. (2.3) *is*: a single number that measures how badly the network’s own top-down model fails to explain its activities. Perception and learning are then two ways of pushing E downhill—by adjusting the activities s_k (inference) or by adjusting the weights W_i (learning), respectively.

2.2.2 Isotropic precision as the modern default

When every precision is the identity, $\Pi_k = I$, the quadratic form in eq. (2.3) collapses to a plain sum of squared errors,

$$E = \frac{1}{2} \sum_{k=1}^n \|\varepsilon_k\|^2. \quad (2.4)$$

This *isotropic* (precision-free) energy is the formulation we adopt for the entire thesis, and it is the mainstream choice in modern PC research on image classification: contemporary benchmark libraries fix the covariance to the identity explicitly rather than learning it (Pinchetti et al., 2024; Millidge et al., 2024). Non-identity precision

is a genuine and theoretically well-motivated feature—a per-layer scalar precision schedule is the lever that helps very deep PC networks train at all (Qi et al., 2025)—but its benefits accrue mainly beyond roughly seven layers and are orthogonal to the questions this thesis asks. For the shallow MNIST-scale networks studied here, $\Pi_k = I$ is both adequate and standard, and we treat eq. (2.4) as the energy of record. We note in passing that an attentive reader will have spotted a notational subtlety: eq. (2.2) indexes predictions in the *feedforward* direction (layer i predicts layer $i + 1$), whereas the neuroscience literature often writes them top-down. The two conventions are identical under an index flip (Millidge et al., 2021); we use the feedforward convention consistently because it is the one the settling dynamics and the kernel of Section 4.3 are written in.

2.3 State-Optimization PC: Settling and the Local Update

We now give the exact algorithm that the rest of the thesis implements, benchmarks, and verifies. “State Optimization” (SO) is the standard predictive-coding formulation: it minimizes the energy eq. (2.4) by gradient descent, alternating two phases. In the *inference* (or *settling*) phase the weights are held fixed and the free states are relaxed toward an energy minimum; in the *learning* phase the states are held at that minimum and the weights take a single gradient step. Because both phases are gradient descent on the *same* energy, the whole scheme is a coordinate-descent on E over the joint variables (s, W) . The presentation here follows the algorithm exactly as we implement it (cf. `pcn/settling.py` and `pcn/model.py`); it is the SO algorithm distilled from Goemaere et al. (2025).

2.3.1 Clamping conventions

Before the dynamics we must say which states are pinned and which are free, because this is the single point most easily got wrong. The input layer s_0 is always clamped to the image x . During *training* the output layer s_n is additionally clamped to the one-hot label y , so only the hidden states settle; the equilibrium configuration is then read off to update the weights. At *test* time only the input is clamped, the output is left free, and the predicted class is the arg max over the settled output state—we let the label node settle rather than supplying it, which is precisely the difference between classifying and merely reconstructing. A third, *generative* convention clamps the label at the top and lets the input settle at the bottom, “painting” an image; whether

this produces digits or noise depends on the training objective, a subtlety we return to in section 2.6.

2.3.2 Feedforward initialization

Settling needs a starting point. We initialize the states by a single feedforward sweep,

$$s_0 = x, \quad s_i = \text{pred}_i = W_{i-1} \phi(s_{i-1}) + b_{i-1} \quad \text{for } i = 1, \dots, n, \quad (2.5)$$

computed sequentially layer by layer. This choice is not arbitrary: copying each prediction into the corresponding state makes every internal prediction error eq. (2.2) exactly zero, so the network starts at the bottom of the “internal” part of the energy landscape with all the residual energy concentrated at the output. As Goemaere et al. (2025) observe, this feedforward heuristic works well because it coincides with the first step of the settling dynamics itself.

2.3.3 The settling dynamics

In the inference phase the weights are fixed and we descend the energy in the states. The gradient of eq. (2.4) with respect to a free state s_k has two contributions, and the structure of these two terms is the entire conceptual content of PC inference. Differentiating,

$$\nabla_{s_k} E = \underbrace{\varepsilon_k}_{\text{own error}} - \underbrace{\phi'(s_k) \odot (W_k^\top \varepsilon_{k+1})}_{\text{error caused below}}, \quad (2.6)$$

where $\odot$ is the element-wise (Hadamard) product. The state then takes a step downhill,

$$s_k \leftarrow s_k - \eta_x \nabla_{s_k} E = s_k - \eta_x \left(\varepsilon_k - \phi'(s_k) \odot (W_k^\top \varepsilon_{k+1}) \right), \quad (2.7)$$

with η_x the state learning rate (also written λ). The two terms have a clean interpretation. The first, ε_k , is the error that layer k itself incurs against the prediction coming from below; moving s_k to reduce it makes the layer better match what was expected of it. The second term is the error that s_k *causes* in the layer above: ε_{k+1} depends on s_k through $\text{pred}_{k+1} = W_k \phi(s_k) + b_k$, and the chain rule produces the transposed weight $W_k^\top$ gated by the local activation derivative $\phi'(s_k)$. Each unit therefore moves so as to simultaneously reduce its own mismatch and the mismatch it propagates upward. Iterating eq. (2.7) for $t = 1, \dots, T$ steps lets the network “settle” into an energy minimum. Two boundary conditions complete the picture:

a clamped state (input always; output during training) is simply not updated, and for the free output layer $k = n$ there is no layer above, so the second term vanishes and $\nabla_{s_n} E = \varepsilon_n$.

A note on implementation that is also conceptually load-bearing: within a single settling step the updates over layers are computed in *parallel* (a Jacobi sweep), because every ε_k and every $\nabla_{s_k} E$ depends only on quantities from the previous step. Only the loop over the T steps and the feedforward initialization are inherently sequential. This parallel-over-layers, sequential-over-time structure is exactly what the fused settling kernel of Section 4.3 exploits, and the state gradient eq. (2.6) is the quantity our tests verify against automatic differentiation.

2.3.4 The signal-decay caveat

A subtlety of the settling dynamics deserves mention, both because it bounds the regime in which SO is well behaved and because it motivates work discussed in Chapter 3. At feedforward initialization all internal errors are zero and only the output error is non-zero. The settling update eq. (2.7) then carries that signal backward through the network at a rate of roughly one layer per step, attenuated at each step by the factor $\eta_x < 1$ needed for stability. The error norm reaching a layer i steps below the output therefore scales binomially, $\|\varepsilon_{n-i}\| \propto \eta_x^i (1 - \eta_x)^{t-i}$, an exponential decay with depth (Goemaere et al., 2025). For small η_x this drives the deep-layer errors below the floating-point floor within a handful of steps, leaving deep layers effectively untrained. For the shallow MNIST networks of this thesis the effect is negligible—two or three hidden layers are simply not deep enough—so SO works well; the decay only becomes a genuine obstacle at depths well beyond what we study. It does, however, refine the folklore on how many settling steps are needed: the safe lower bound is $T \geq L$ for an L -layer network (Qi et al., 2025), with the upper bound governed by the decay rather than by a fixed multiple of depth.

2.3.5 The local Hebbian weight update

Once the states have settled to (approximate) equilibrium we hold them fixed and take one gradient step in the weights. The gradient of eq. (2.4) with respect to W_i is, by eq. (2.2),

$$\nabla_{W_i} E = -\varepsilon_{i+1} \phi(s_i)^\top, \quad (2.8)$$

so the weight update is

$$W_i \leftarrow W_i + \eta_w \varepsilon_{i+1} \phi(s_i)^\top, \quad (2.9)$$

with η_w the weight learning rate (kept separate from η_x throughout). This is the heart of the whole approach. The update for W_i requires *only* the prediction error of the layer above, ε_{i+1} , and the activity of the layer below, $\phi(s_i)$ —both quantities local to the synapse W_i connects. It is a Hebbian outer product of a presynaptic activity and a postsynaptic error: there is no global backward pass, and no chain of transposed weight matrices threaded across the whole network (Whittington and Bogacz, 2017; Bogacz, 2017). The weight-transport problem that makes backpropagation biologically implausible simply does not arise. This locality is what the term “local learning” in the title of this thesis refers to, and recovering it on a parallel accelerator without sacrificing speed is the engineering problem Section 4.3 attacks.

2.4 Relation to Backpropagation

Predictive coding and backpropagation are not unrelated algorithms that happen to solve the same task; under a precise and well-studied assumption, the local PC update of eq. (2.9) *computes the same gradient* that backpropagation would. Understanding exactly when this holds is essential to interpreting our comparison results honestly, so we state it carefully.

2.4.1 The fixed-prediction equivalence

The foundational result is due to Whittington and Bogacz (2017): a supervised predictive coding network with value nodes, error nodes, transposed-weight feedback gated by the activation derivative, and the local Hebbian rule eq. (2.9) *approximates* the backpropagation gradient, and does so without any non-local plasticity. Millidge et al. (2022) generalized this from layered MLPs to arbitrary computation graphs. The mechanism is the following. If the network is at the feedforward state—the configuration in which every internal error is zero and only the output error is non-zero—then the settling update eq. (2.7), run for a short time, transports the output error backward through exactly the transposed weights $W_k^\top$ gated by $\phi'(s_k)$ that backpropagation uses. Under this “fixed-prediction” assumption, the prediction errors ε_k at the layers come to equal the backpropagated error signals, and the local update eq. (2.9) reproduces the BP weight gradient. In the limiting case the correspondence is exact (Whittington and Bogacz, 2017). A PC network can therefore learn the same thing as an MLP, but by minimizing free energy rather than by applying the chain rule.

2.4.2 Prospective configuration is the equilibrium, not a separate algorithm

This equivalence is sometimes obscured by the framing of “prospective configuration” (Song et al., 2024). In that framing, a PC network first changes its neural activity across the network so that the output neurons better predict the target, and *then* consolidates that activity change into the weights—the reverse of backpropagation’s weights-first, activity-follows ordering. It is important to be clear that prospective configuration is not a new learning rule grafted onto PC: it is simply a name for the behavior of standard energy-based PC when the activities are allowed to relax to equilibrium before the weights move. The inference-before-learning two-phase structure of section 2.3 *is* prospective configuration. The substantive empirical claim attached to the term—that this ordering helps in regimes such as online, continual, and few-shot learning—is a separate matter that we test directly and report, with appropriate caveats, in Sections 5.3 and 5.4.

2.5 When Does PC Equal BP?

Given that PC can recover the BP gradient under the fixed-prediction assumption, the natural question—and the one that frames our first research question—is the converse: *when does PC do something genuinely different from BP?* The answer determines whether predictive coding is merely a biologically flavored re-derivation of backpropagation or a distinct learning algorithm with its own behavior, and it must be stated with care to avoid overclaiming in either direction.

2.5.1 The provable equivalence regimes are narrow

The conditions under which PC provably converges to BP are restrictive. The most recent analysis, Innocenti et al. (2026), shows that the PC energy with equilibrated activities converges to the quadratic backpropagation loss and yields the same gradients—but only for *linear* residual networks, in the regime of *width far greater than depth*, and only when the activity equilibrium is genuinely reached. This is emphatically *not* an unconditional “PC = BP” statement; it is a careful limit theorem hedged by three structural conditions, and we cite it only with those conditions attached. Outside that regime—nonlinear activations, finite width comparable to depth, or settling stopped short of equilibrium—there is no guarantee that the PC gradient equals the BP gradient, and indeed the two can differ.

2.5.2 The role of the settling budget

There is a complementary way to see when the two coincide, phrased in terms of how much settling is allowed. If the supervision signal is infinitesimally small, or the settling is stopped almost immediately after feedforward initialization, the activities never move substantially away from the feedforward state, the fixed-prediction assumption holds tightly, and PC reduces to BP (Song et al., 2024). Any behavior in which PC differs from BP therefore requires *substantial* settling: enough relaxation steps, and a large enough state learning rate, that the activities genuinely reconfigure before the weights move. Conversely, a PC experiment run with a tiny settling budget is measuring backpropagation in disguise and should be expected to show no PC-specific effect. This is a methodological razor we wield repeatedly in Chapter 5: a null result for PC is only informative if the settling was real. The open empirical question, then—the framing for RQ1—is whether, in the regimes where PC *can* differ from BP, it actually does so to any measurable and reproducible degree on standard image-classification tasks. A sober reading of the literature is that the advantages reported for PC have not been consistently reproduced (Zahid et al., 2023), which is precisely why a fair, decounfounded comparison (Sections 4.4 and 5.3) is worth doing.

2.6 Generative versus Discriminative PC

A final distinction is needed because it governs what a PC network can and cannot do, and it is easy to assume more than the architecture delivers. The distinction is one of *orientation*: which end of the network is treated as the latent cause and which as the observation.

2.6.1 Discriminative orientation

A *discriminative* PC network is oriented image→label: the input layer holds the image, the output layer holds the label, and predictions flow toward the label. This is the supervised classifier studied for most of this thesis, and it is what the clamping conventions of section 2.3 describe. Such a network classifies well, but it cannot generate: clamping a label at the output and letting the input settle does not paint a digit, because the network was never trained to predict pixels from a class. Doing so produces noise, not an image—a point we verify empirically rather than assume.

2.6.2 Generative orientation

A *generative* PC network reverses the orientation, label→image: a low-dimensional latent or label sits at the top and the image is predicted at the bottom, so the layer sizes run, for example, $10 \rightarrow 256 \rightarrow 256 \rightarrow 784$. Trained with the image clamped at the bottom and the latent free at the top, this network learns a top-down generative model of the data. The very same settling machinery of section 2.3 then supports a family of capabilities for free: clamping a class and settling the image produces per-class prototypes; clamping the visible pixels and settling the occluded ones performs inpainting; and the residual energy of a settled input, $\frac{1}{2} \sum_k \|\varepsilon_k\|^2$, serves directly as an anomaly score, since an input the model explains poorly leaves high residual energy. That a single energy and a single settling rule yield classification, generation, inpainting, and anomaly detection simply by changing what is clamped is one of the conceptually attractive features of predictive coding, and the capability demonstration in Section 5.5 builds directly on this orientation distinction.

Chapter 3

Related Work

This chapter surveys the prior art against which the three contributions of this thesis must be measured, and from that survey it establishes precisely where the present work sits. Its purpose is positioning rather than methodology: Chapter 2 has already introduced the State-Optimization formulation of Predictive Coding (PC) that the thesis studies, and Chapter 4 will describe what was actually built. Here the concern is narrower. A central claim of this thesis is a systems claim — that its fused CUDA settling kernel is, to our knowledge, the first hand-written kernel of its kind for PC — and a claim of priority is only as strong as the audit of prior work that backs it. We therefore devote the bulk of this chapter to a careful, source-by-source examination of the implementations that could plausibly contain such a kernel, of the algorithmic family (Equilibrium Propagation) that is PC’s closest relative, and of the systems-level technique (persistent, single-launch GPU kernels) that the kernel borrows and re-applies. We then turn to the continual-learning literature, because two of the thesis’s empirical findings — on forgetting and on the stability–plasticity trade-off — can only be interpreted against the metrics and methods established there. We end with a single section of synthesis that states plainly what is, and what is not, being claimed.

A word on tone is in order before we begin. The defining commitment of this project is honesty about the strength of its claims, and nowhere is that more important than in a related-work chapter that grounds a priority claim. A negative-existence claim — “no prior implementation ships such a kernel” — can never be proved absolutely, because no audit can index every private or unpublished codebase. We therefore frame the systems claim narrowly, qualify it with “to our knowledge” throughout, and are explicit about the prior art that the kernel’s *technique* draws on, claiming novelty only for its *application* to the PC settling loop. The reader should come away convinced not by breadth of assertion but by the directness of

the underlying inspection.

3.1 Predictive Coding Implementations and Libraries

Predictive Coding has, over the last half-decade, acquired a small but genuine ecosystem of software. Before asking whether any of these tools accelerates the settling loop with a hand-written kernel, it is worth recalling *why* settling is the natural place to look for such an effort. The defining feature of State-Optimization PC, set out in Section 2.3, is that inference is not a single feedforward pass but an iterative relaxation: for each input the layer states s_k are updated for T steps along the negative gradient of the energy $E = \frac{1}{2} \sum_k \|\varepsilon_k\|^2$, with errors $\varepsilon_k = s_k - \text{pred}_k$ and predictions $\text{pred}_{i+1} = W_i \phi(s_i) + b_i$. This relaxation is the computational bottleneck of PC and the part of the algorithm that maps least gracefully onto the matrix-multiply hardware for which backpropagation was tuned. It is, in other words, exactly where one would write a custom kernel if one were going to. The question of this section is empirical: has anyone done so?

3.1.1 The two implementation idioms: JAX/JIT and PyTorch-manual

The mature PC libraries fall cleanly into two families distinguished by how they execute the settling loop. The first idiom is JAX, in which the relaxation is expressed in terms of high-level array operations and handed to the XLA compiler via `jit`, sometimes with `vmap` for batching or an ODE solver for the continuous-time view. Pinchetti et al. (2024) present PCX (the `pcax` package), a JAX framework built on `optax` that has become a reference benchmark suite for the field; its companion paper is candid that the JIT compiler is “unable to parallelize the executions of the layers” and that the `vmap` remedy applies only “in the unpractical case where all the layers have the same dimension.” Millidge et al. (2024) present JPC, a compact JAX/Equinox library (under a thousand lines of core code) that casts settling as an ODE and integrates it with Difffrax, reporting that a second-order Heun integrator settles faster than Euler. The `ngc-learn` library of the NAC Lab and the depth-scaling work of Innocenti et al. (2026) (the μ PC parametrisation, layered on PCX) likewise live in JAX. In all of these, the settling loop is compiled to XLA or solved by Difffrax; there is no separate, hand-written device kernel, and the burden of acceleration rests entirely on the compiler.

The second idiom is PyTorch used as a tensor engine, with the settling iteration written out explicitly as a Python loop over analytic tensor operations or, in the autograd-based variants, as repeated calls to `backward()` on the energy. van Zwol et al. (2024)’s PRECO implements PC and graph-structured PC networks this way, with the per-step state updates expressed as manual matrix operations inside a Python `for-t` loop. Torch2PC (van Zwol et al., 2024) computes the updates via autograd vector–Jacobian products and offers an `Exact` mode whose gradients reproduce those of backpropagation, which makes it a useful numerical cross-check but not an accelerated path. The research scripts `pypc` and `pybrid` follow the same manual-tensor pattern, as does the public code accompanying Song et al. (2024)’s Prospective Configuration study. Crucially for the present thesis, the error-based reparametrisation of Goemaere et al. (2025) (ePC), which we discuss further in Chapters 6 and 7 as the most closely related attack on the same end goal of fast deep settling, is itself a PyTorch/Lightning implementation that settles by calling `E.backward()` and stepping an optimiser. ePC accelerates PC *algorithmically* — by optimising over the prediction errors rather than the states, so that learning signals reach all layers at once instead of decaying exponentially with depth — not at the kernel level. It is therefore orthogonal to, and in principle combinable with, a faster kernel; it must be cited and distinguished precisely so that it is not mistaken for an overlooked instance of the systems claim.

3.1.2 The audit: eleven libraries, zero settling kernels

The grounding for this thesis’s priority claim is a direct source audit of every PC library named above, summarised here from the verification record. For each repository the audit cloned the source (or, where rate limits intervened, inspected the file tree through the GitHub trees API) and searched both for CUDA source files (`.cu`, `.cuh`, `.cpp`) and for the textual signatures of just-in-time kernel compilation (`cpp_extension`, `load_inline`, `__global__`, `nvcc`, `CUDAExtension`, `triton.jit`). The result is uniform: across PCX, JPC, `ngc-learn` (modern JAX and the TensorFlow-era `ngc-learn-legacy`), μ PC, PRECO, Torch2PC, `pypc`, `pybrid`, ePC, and Song’s code, there is not a single hand-written CUDA settling kernel. Settling runs everywhere as a Python loop over framework operations, as a JAX/XLA JIT compilation, or as a Diffrax ODE solve — never as one fused custom CUDA kernel. The only apparent matches were false positives: a Python file named `expKernel.py` in `ngc-learn` (a mathematical, not device, kernel) and the C sources of `ngc-learn-legacy`, which turn out to be an INFIMNIST data generator unrelated to settling. The strength of this sub-claim rests on its directness — it is

a matter of inspection rather than inference — and it is corroborated by PCX’s own published admission that its inference cannot be fully parallelised. This is the empirical foundation for the first contribution’s framing: the gap the kernel fills is real, and it is observable in the source trees of the field’s own tools.

3.2 Equilibrium Propagation: The Closest Algorithmic Relative

Predictive Coding does not stand alone. It belongs to a broader family of energy-based learning algorithms whose common shape is a two-phase procedure: relax a network of units to a fixed point of some energy function, then update the weights using only quantities local to that fixed point. The closest relative of PC within this family is Equilibrium Propagation (EP), introduced by Scellier and Bengio (2017). EP, like PC, settles a network to equilibrium (its “free phase”), then nudges the outputs toward the target and settles again (its “nudged phase”), and the difference between the two equilibria furnishes a local, backprop-approximating weight update. The structural parallel to PC’s inference-then-plasticity schedule is exact enough that any audit underwriting a “first for PC” kernel claim must also rule out the EP side: if some EP implementation already shipped a hand-written settling kernel, the claim would be substantially weakened by analogy, even though EP is a different algorithm. This section reports the second half of the audit, which closes that gap.

3.2.1 The canonical and recent EP implementations

The audit enumerated the canonical EP lineage and its more recent descendants. The original implementation of Scellier and Bengio (2017) is written in Theano. The influential variants — the symmetric-nudging and holomorphic refinements associated with Laborieux et al. (2021), the recurrent-backprop-equivalence work of Ernoult and colleagues, and the agnostic-EP and spiking-EP reimplementations — are written in PyTorch with the settling phases driven by autograd, by Euler iteration of explicit tensor operations, or by a Python host loop. Laborieux et al. (2021)’s holomorphic variant moves to JAX with Haiku but still settles in an eager `while` loop rather than a compiled scan or a Pallas/Triton kernel. The recent end of the literature (for example the 2025–2026 stochastic-EP and intermediate-error-signal scaling papers) is explicit in its own text about running “in Python using the PyTorch framework” on commodity GPUs. Across roughly two dozen repositories spanning the canonical authors, community reimplementations, and ad-

jacent energy-based models (modern Hopfield networks, the Energy Transformer), the kernel signatures (`__global__`, `cpp_extension`, `load_inline`, `triton.jit`) return empty or false-positive (an EDA-placement tool and a lattice-QCD project that happen to use the string `eqprop`).

3.2.2 The single near-miss, and why it does not refute the claim

One repository deserves to be singled out because it comes closest. A multi-algorithm bio-plausible-learning framework (`autonull/bioplausible`) does ship hand-written GPU kernels for EP dynamics. It does not, however, satisfy any of the three features the thesis claims. First, it is EP, not PC — a related but distinct algorithm, so “first for PC” remains untouched. Second, its kernels are written in Triton, not hand-written CUDA; the repository contains no `.cu/.cuh` sources and no `__global__` or `cpp_extension` usage. Third, and most importantly, the kernels are not a fused, persistent, states-resident in-kernel settling loop: each kernel computes a single elementwise relaxation step of the form $h \leftarrow (1 - a)h + a \tanh(\text{pre-activation})$, the matrix multiply is *not* fused in, and the settling iteration itself runs on the host in a Python loop. (The audit also flagged a naming trap in the same repository: a `fused_settle_step` living under a `cuda` path whose body is in fact pure PyTorch operations, with a docstring that concedes “PyTorch still launches separate kernels.”) This repository is the nearest EP-side prior art and should be cited defensively, but it weakens nothing: it is the wrong algorithm, the wrong implementation technology, and the wrong execution structure. The conclusion of the EP audit is therefore that the closest algorithmic relative *hardens* rather than refutes the priority claim: to our knowledge no EP implementation ships a hand-written CUDA settling kernel either.

3.2.3 A scope caveat: analog and neuromorphic acceleration

For completeness, and in the interest of not overstating the gap, it must be said that EP *does* have a vigorous acceleration literature — it simply lives in physical, not digital, hardware. Analog-circuit, coupled-oscillator, memristor, and spiking implementations exploit EP’s defining property that the free-phase relaxation is itself a physical energy-minimisation process, with reported speed-ups of two orders of magnitude. This is precisely the regime in which energy-based learning is most naturally at home, and it is worth acknowledging plainly. It is not, however, prior art for a GPU-kernel claim: these are physical devices, not software settling kernels,

and the present thesis makes no claim about them. The novelty being asserted is specific to running the PC settling loop on a conventional digital GPU.

3.3 Systems for Iterative / Fixed-Point Inference

The argument so far establishes that no PC or EP implementation ships a hand-written settling kernel. It would be a serious overstatement, however, to conclude that the *technique* embodied in such a kernel is novel. It is not, and intellectual honesty requires that this be stated as forcefully as the gap itself. This section places the kernel’s design squarely within an established systems lineage and identifies what is, and is not, the thesis’s own contribution at the systems level.

3.3.1 The persistent, single-launch kernel as prior art

The pattern at the heart of an efficient settling kernel — keep the network’s state resident on-chip and run the entire iterative loop inside a single kernel launch, rather than issuing one launch per step — is canonical GPU-engineering prior art, and it predates this work by nearly a decade. Its archetype is the persistent recurrent-neural-network kernel of Diamos et al. (2016), also exposed in cuDNN as the `CUDNN_RNN_ALGO_PERSIST_STATIC/DYNAMIC` algorithms. There a single launch keeps its thread blocks resident across all recurrent timesteps, holds the weights and activations in the streaming-multiprocessor register file and shared memory, and synchronises between steps with a global barrier — a design motivated explicitly by the observation that per-step kernel-launch overhead dominates at small batch (the persistent kernel reports order-of-magnitude speed-ups over a per-step cuBLAS baseline at mini-batch four). The same engineering pattern has reappeared more recently in the LLM “megakernel” literature, in which an entire forward or decode pass is fused into one persistent, state-resident kernel to win back small-batch launch overhead. The thesis’s kernel is the same pattern; it borrows it deliberately. What it claims is the *first transfer of that pattern to the PC settling loop* — together with the specific layer-parallel, T -sequential decomposition and the precision-weighted, ϕ' -gated bidirectional update that the PC dynamics require — not the invention of the pattern.

3.3.2 Deep Equilibrium models and the conceptual neighbours

A second relevant systems lineage is that of fixed-point models that, like PC, define their output as the solution of an equilibrium condition rather than the result of a fixed feedforward depth. Deep Equilibrium models (Bai et al., 2019) are the canonical example: they solve for a fixed point of a layer using root-finders such as Anderson acceleration or Broyden’s method, and back-propagate through the equilibrium using the implicit function theorem. DEQ is the nearest *conceptual* relative on the fixed-point axis, and ePC (Goemaere et al., 2025) is the nearest relative on the energy-minimisation axis; both attack the same end goal of fast fixed-point inference. Neither, however, does so with a hand-written CUDA settling kernel — DEQ solves entirely in PyTorch, and ePC, as noted above, reparametrises the problem rather than rewriting the kernel — so neither refutes the systems claim. They do, however, define the conceptual neighbourhood in which the contribution should be read.

3.3.3 The framework tax: a generic observation, not a contribution

Finally, the empirical phenomenon that the kernel exploits — that an inference loop built from many small framework operations becomes launch-overhead-bound at small and medium batch sizes — is itself a generic, already-published observation about deep-learning frameworks, sometimes called the “framework tax.” PCX’s own paper alludes to it when it laments the cost of “multiple smaller operations.” This thesis measures that effect for the specific case of PC settling and uses it to motivate the kernel, but it does not and must not claim the observation itself as a contribution. The honest framing, which Chapter 6 develops, is that the kernel is a faithful *measurement* and *first application* of a known technique to a known kind of bottleneck, not the discovery of either. This careful attribution is what allows the priority claim to survive scrutiny: it is narrow, PC-specific, qualified, and explicit about its debts.

3.4 Continual Learning

The thesis’s empirical contributions are not confined to the kernel. Following the fair PC-versus-backpropagation methodology that Section 4.4 formalises, two of its findings concern *continual learning* — the setting in which a model must learn a

sequence of tasks without revisiting earlier data. This setting is where Predictive Coding is most often conjectured to hold an advantage over backpropagation, and so it requires its own grounding in prior work, both to define the metrics by which any such advantage would be measured and to position the baselines against which it is compared.

3.4.1 Catastrophic forgetting and the stability–plasticity trade-off

The core difficulty of continual learning is *catastrophic forgetting*: when a network trained by gradient descent learns a new task, the same weight updates that fit the new data overwrite the representations supporting earlier tasks. This is one face of a more general tension known as the stability–plasticity trade-off. A model must be *plastic* enough to acquire new tasks and *stable* enough to retain old ones, and methods that improve retention frequently do so by reducing the rate at which new tasks are learned. This trade-off is not a confound to be eliminated but a curve along which methods sit, and it is essential to interpreting any claim that one learning rule “forgets less” than another: forgetting less while also learning each task less well is movement *along* the curve, not a strict improvement *over* it. The conjectured appeal of PC in this setting, articulated most forcefully by Song et al. (2024), is that its inference phase settles the hidden units into a “prospective” configuration before plasticity acts, which is argued to distribute the error in a way that avoids the weight interference responsible for forgetting — the difference between altering activity to anticipate a side effect and back-propagating a shared-weight update that damages a correct, unrelated association.

3.4.2 Measuring forgetting: the GEM metrics

To compare methods on this axis at all, one needs agreed metrics, and the standard set comes from the Gradient Episodic Memory work of Lopez-Paz and Ranzato (2017). From the matrix of test accuracies R_{ij} — the accuracy on task j after training through task i — one derives *average accuracy* (overall performance at the end of training), *backward transfer* (BWT, the influence of later learning on earlier tasks, whose negative is the amount forgotten), and *forward transfer* (the influence of earlier learning on yet-unseen tasks). These metrics are what make a statement such as “PC forgets less” precise and falsifiable, and the thesis’s continual-learning experiments report them directly. Lopez-Paz and Ranzato (2017)’s GEM itself is a replay-style method that stores a small episodic memory of past examples and

constrains new gradients not to increase the loss on that memory; it is included here as the origin of the evaluation framework rather than as a baseline the thesis re-implements. The broader infrastructure of the field — standardised benchmark streams such as Split- and Permuted-MNIST and the metric implementations that accompany them — is consolidated in the Avalanche library (Lomonaco et al., 2021), which the thesis draws on for benchmark construction even where it computes the metrics independently.

3.4.3 Regularisation baselines: Elastic Weight Consolidation

The most directly relevant baseline is a regularisation method rather than a replay method. Elastic Weight Consolidation (EWC) of Kirkpatrick et al. (2017) reduces forgetting without storing past data by adding a quadratic penalty that anchors each weight to its post-task value, with a per-weight stiffness given by the diagonal of the Fisher information matrix — in effect, weights that mattered for previous tasks are made expensive to move. EWC is the canonical answer to “how do we forget less while staying within standard backpropagation,” and it therefore makes the natural reference point for any claim that PC’s inference phase confers a forgetting advantage. Reading PC’s behaviour against EWC — and against a fair, loss-matched backpropagation baseline — is what keeps the stability–plasticity trade-off honest: it situates PC on the same curve as an explicit anti-forgetting mechanism rather than against a naive, untuned baseline. Chapter 5 reports the resulting comparison, and Chapter 6 interprets it in light of exactly the trade-off articulated here.

3.5 Positioning of This Work

Against this backdrop the position of the thesis can be stated in a single, deliberately modest paragraph. This is a from-scratch, reproducible study of State-Optimization PC on MNIST-scale data, contributing three focused and independently verified results rather than a new algorithm or a frontier-scale system. First, a systems contribution: a hand-written, fused, single-launch CUDA settling kernel that holds the layer states resident across all T steps and that is, to our knowledge, the first such kernel for the PC settling loop — a claim grounded in the source audit of Chapter 3 of eleven PC libraries and the Equilibrium Propagation family, and bounded explicitly by the persistent-RNN and megakernel prior art whose technique it re-applies rather than invents. Second, a methodological and empirical contribution:

a fair, loss-matched, multi-seed comparison of PC against backpropagation, whose headline finding is largely *null* — vanilla PC neither clearly beats nor clearly trails a fairly tuned, loss-matched backpropagation baseline at this scale — and whose value lies in the methodology and in disentangling three confounds (the loss function, the stability–plasticity trade-off, and an untuned baseline) that can manufacture spurious wins in either direction. Third, a set of demonstrations of PC’s distinctive capabilities — generative reconstruction, inpainting, and anomaly detection — and of its behaviour under variants such as incremental PC and continual-learning regularisers. What is *not* claimed is as important as what is. The thesis does not claim to solve the field’s open problems — the depth-scaling failure, the PC-equals-backprop puzzle, or the scale gap (Innocenti et al., 2026) — which remain frontier-laboratory work that a single-author MNIST study can reproduce and characterise but not resolve. It does not claim a general-purpose kernel, a general speed-up, novelty for the persistent-kernel technique, nor the framework-tax observation as its own. And it does not claim a PC advantage over backpropagation that its own honestly reported confidence intervals do not support. The contribution is the rigour of the reproduction and the soberness of the framing; the survey above is what licenses both.

Chapter 4

Method and Implementation

This chapter describes *how* the study was built: the from-scratch State-Optimization predictive-coding (SO-PC) network and the single interface through which every experiment runs; the correctness regime that lets us trust hand-derived updates; the fused CUDA settling kernel that is the project’s principal systems contribution; the fair-comparison protocol that disciplines the predictive-coding-versus-backpropagation (PC-vs-BP) question; the continual-learning harnesses (including a Song-exact alternating replication); the generative variant of the same machinery; the optional incremental-PC and elastic-weight-consolidation extensions; and the reproducibility tooling. Throughout, the emphasis is on design decisions and their justification. Quantitative outcomes are deferred to Chapter 5; the underlying PC mathematics is developed in Section 2.3 and is referenced here rather than re-derived.

A single conviction shapes every choice below. The point of the project is to *understand and demonstrate* a learning rule that is local — one in which every parameter and state update can be computed from quantities available at the two layers it connects, with no global error signal propagated backward through the network. We therefore use PyTorch (Paszke et al., 2019) purely as a tensor engine: there is no call to `loss.backward()` anywhere on the PC path, and every state and weight update is performed by hand under `@torch.no_grad()`. Autograd appears in exactly two places, both deliberate: as an external oracle in the gradient-correctness tests (Section 4.2), and as the learning rule of the backpropagation baseline against which PC is measured. This is not an aesthetic preference. If autograd silently supplied a gradient anywhere in the settling or learning loop, the claim “this is local learning” would be hollow, and the comparison with backprop would be circular. Keeping the PC arm autograd-free is what makes the study honest.

4.1 From-Scratch PCN Architecture and Interface

4.1.1 The model object

The network is a small object, `PCN`, holding a list of weight matrices W_i , bias vectors b_i , an activation function ϕ together with its derivative ϕ' , and bookkeeping for device and dtype. For a topology with layer sizes $[n_0, n_1, \dots, n_L]$ (input n_0 , output n_L) there are L weight matrices, where we write $L = n$ for the number of layers/-transformations in the code. The single primitive the rest of the system relies on is the per-layer prediction

$$\text{pred}_{i+1} = W_i \phi(s_i) + b_i,$$

implemented as `predict(i, state)`. Everything else — feedforward initialisation, settling, the local weight update, evaluation, the kernel, the baselines — is expressed in terms of this one function, so that the forward computation is defined in exactly one place and cannot drift between the PC arm and its backprop control.

Weights are initialised orthogonally by default. This is a substantive choice rather than a default inherited from convention: orthogonal initialisation keeps the layer-to-layer Jacobian eigenvalues near one, which mitigates the exponential signal decay that afflicts deep SO-PC during settling (Section 2.3 and the depth discussion below). The activation is `tanh` by default, with `identity` and `sigmoid` also supported; `sigmoid` matters because it is the activation Song et al. use in the regime we replicate (Section 4.5).

4.1.2 Feedforward initialisation, settling, and the local update

Training proceeds per minibatch in four conceptual stages, matching the SO algorithm of Section 2.3:

1. **Feedforward init.** The input is clamped at $s_0 = x$ and the remaining states are filled by a single forward pass, $s_{i+1} \leftarrow \text{pred}_{i+1}(s_i)$. By construction this sets every internal error $\varepsilon_k = s_k - \text{pred}_k$ to zero, so settling begins from a zero-internal-energy state and only the clamped output injects a non-zero error. This is not merely a convenient warm start: it coincides with the first step of the error-optimization view of PC, which is part of why it is such a good initialiser.

2. **Clamp.** The input node is clamped to the image and (during training) the output node is clamped to the one-hot label.
3. **Inference / settling.** With the weights held fixed, the free states are relaxed for up to T steps toward the energy minimum. Each step is a synchronous (Jacobi) update: all errors ε_k are computed first, then all free-state updates $s_k \leftarrow s_k - \eta_x (\varepsilon_k - \phi'(s_k) \odot (W_k^\top \varepsilon_{k+1}))$ are applied from those errors, so that no layer’s update sees another layer’s change within the same step. The output layer, when free at test time, updates with ε_k alone.
4. **Learning.** At the settled equilibrium the weights are updated by the purely local Hebbian rule $W_i += \eta_w \varepsilon_{i+1} \phi(s_i)^\top / B$ and $b_i += \eta_w \overline{\varepsilon_{i+1}}$, with B the batch size. Each update uses only the post-synaptic error and the pre-synaptic activation of the layer it belongs to.

The settling routine is deliberately compact — on the order of fifty lines — because it is the conceptual core of the repository and must be auditable line by line. It takes flags that, in the same few lines, support the discriminative training case (input and output clamped, hidden states free) and the generative / inpainting / anomaly cases (input free or partially masked), which is what lets the generative experiments reuse exactly the same dynamics (Section 4.6).

4.1.3 The convergence criterion

By default settling runs a fixed number of steps T . We added an optional tolerance: when a relative energy-change threshold `tol` is supplied, the loop stops once $|E_t - E_{t-1}| / (|E_{t-1}| + \epsilon) < \text{tol}$. With `tol = None` the original fixed- T behaviour is reproduced exactly. This small addition turns “how many steps does settling need to converge?” into a measurable quantity rather than a hyperparameter we assume. We use it as a diagnostic: `settling_steps_to_converge` is measured on a training batch with a generous step cap (five times T , at least one hundred) and a reference tolerance of 10^{-3} , so the reported value is the true number of steps the dynamics need to plateau, independent of the configured T . If that number exceeds the training T , the network is under-settling — directly relevant because PC’s claimed advantages are predicated on reaching the activity equilibrium (Song et al., 2024; Innocenti et al., 2026; Qi et al., 2025).

4.1.4 The `train_and_eval(config)` contract

The only deliberate abstraction in the system is a single function, `train_and_eval(config)` $\rightarrow$ `metrics`. It accepts a configuration dictionary — hidden sizes, T , the state and weight learning rates, activation, weight init, number of epochs, batch size, seed, backend, and so on — builds a PCN, trains it on MNIST (LeCun et al., 1998), and returns a metrics dictionary containing test accuracy, an energy curve (mean equilibrium energy per epoch), noise-robustness (accuracy across input-noise levels), training wall-clock, the settling-steps-to-converge diagnostic, and the fully resolved configuration. There is no plugin or adapter architecture: a single coherent implementation with one mechanism, exposed through one interface. The autonomous hyperparameter search (Section 4.8) and the experiment harnesses are thin layers that call this function and log the result — no rewrite, by design.

Two small details in the contract guard against silent error, in keeping with the honesty principle. First, the thesis-specification names η_x and η_w are accepted as configuration aliases for the internal `lr_state` and `lr_weight`, with the spec names taking priority, so a config written to the documented notation is honoured rather than silently ignored. Second, unknown keys raise a warning instead of being dropped, and reserved-but-unimplemented features (non-isotropic precision schedules; update variants other than the implemented two) raise `NotImplementedError` rather than silently no-op'ing. The default weight learning rate was raised from 10^{-3} to 10^{-2} after the lower value was found to underfit MNIST; precision is hard-coded to the isotropic case $\Pi = I$, which reduces the energy $E = \frac{1}{2} \sum_k \|\varepsilon_k\|^2$ to a plain sum of squared errors and is mathematically exact for that choice.

4.2 Correctness: Hand-Derived Gradients vs. Autograd

Because no autograd runs on the PC path, the burden of proof falls entirely on us: the settling update and the Hebbian weight update are gradients of the PC energy that we derived and transcribed by hand, including the easily-mistaken signs, the transpose on the feedback term $W_k^\top \varepsilon_{k+1}$, and the $\phi'(s_k)$ gating factor. A sign error or a missing transpose would not crash — it would quietly train a slightly different objective and contaminate every comparison. We therefore treat correctness as a first-class deliverable and pin it down two ways.

First, against an analytic reference. The exact SO update — signs, transposition, and the ϕ' gating — is validated against an independent analytic NumPy derivation,

and the implementation is held to match it; this is a hard convention of the project, not a one-off check. Second, against autograd as an external oracle. In the test suite (and only there) we build the same energy with autograd enabled and confirm that the hand-coded settling-state gradient and the hand-coded weight update agree with `torch.autograd` to roughly 10^{-5} . This is the one legitimate use of autograd on the PC side: not to *compute* the updates the network learns by, but to *certify* that our local rules equal the true energy gradients. A complementary property test asserts that the PC energy decreases monotonically under settling on toy tensors, so that a regression in the dynamics is caught even when the analytic check passes. The tests are hermetic — they run on small synthetic tensors with no MNIST download and complete on CPU — so that correctness is cheap to re-verify on any machine (Section 4.8). The exact derivations these checks target are given in Appendix A.1.

4.3 The Fused CUDA Settling Kernel

4.3.1 Motivation: predictive-coding inference is launch-overhead-bound

The defining cost of PC, relative to a feedforward network, is the iterative inference phase: each training (or test) example requires T settling steps before any weight update. On hardware optimised for the single dense matmul pattern of backprop, this iterative structure is a mismatch. Crucially, within a single settling step the work is not one large operation but a stream of *small* ones — a small matmul to form each prediction, an elementwise subtraction for each error, a transpose-matmul for the feedback term $W_k^\top \varepsilon_{k+1}$, the elementwise ϕ' gating, and the state update — and these recur for every layer and every step. At MNIST widths (input 784, hidden 256) each of these operations is tiny, so the dominant cost is not arithmetic but the fixed overhead of launching a GPU kernel for each of them. A standard PyTorch settling step issues many such launches; we measure exactly 31 kernel launches per step on our two-hidden-layer topology, independent of width, so that $T = 20$ costs 620 launches and $T = 80$ costs 2480. This is the “framework tax”: the wall clock is set by launch count, not by floating-point work. It also explains the low GPU utilisation (roughly a quarter to a half) we observe for the PyTorch PC arm.

The opportunity is precisely the parallel structure of one settling step. Within step t , all errors ε_k depend only on the states at t and can be computed together; then all free-state updates Δs_k depend only on those errors and can be applied together. Only the chain of T steps, and the feedforward initialisation, are inherently

sequential. A handwritten kernel can therefore fuse an entire step — or the whole T -step loop for a batch — into a single launch, holding the states resident in shared memory or registers across all T steps and reading the weights from global memory once (persistent-kernel style). The number of launches collapses from hundreds or thousands to *one*.

4.3.2 Novelty and honest scope

We audited the open-source landscape to place this contribution honestly. Across eleven open PC libraries there is no handwritten CUDA inference kernel: they are uniformly JAX/JIT-based or built on PyTorch autograd, and contain zero `.cu` files. We extended the audit to the closely related Equilibrium Propagation family (eighteen repositories) and found the same gap, the nearest neighbour being a per-step Triton kernel driven by a host-side loop. The defensible claim is therefore narrow and specific: *the first handwritten fused CUDA settling kernel for predictive coding, to our knowledge*. We are equally explicit about what is *not* novel. The fusion technique itself — persistent, state-resident, single-launch kernels — is established prior art from persistent RNNs and large-language-model megakernels, and the “launch-overhead-dominated” observation is a generic framework property, not our discovery. Our contribution is the transfer of that technique to PC plus a clean measurement of how much of PC’s inference overhead is launch overhead. The most recent algorithmic neighbour, error-optimization PC (Goemaere et al., 2025), addresses settling efficiency at the algorithm level rather than the kernel level and is cited as the contrasting approach. The kernel audit and verdict are documented in full as part of the project record.

4.3.3 The fused design and its three versions

The CUDA source is the single source of truth: a real `.cu` file compiled at first use via `torch.utils.cpp_extension.load` (JIT, cached), not an inline string or a commented stub. The C++/CUDA extension exposes the kernel through a `pybind11` module in the same file. Three versions of the two-hidden-layer kernel evolved, each addressing a specific bottleneck revealed by measurement:

v1 — one block per sample. Each thread block settles one sample, holding that sample’s states, errors, and activations in shared memory across all T steps. This eliminates the launch overhead and wins decisively at small batch. Its weakness is that every block re-reads the full weight matrices from global

memory each step, so at large batch it is bandwidth-bound: with one block per sample the same weight element is fetched once per sample.

v2 — one block per tile of samples. To recover the lost weight reuse — the key optimisation of any GEMM — v2 assigns a tile of $TB=8$ samples to each block and reads each weight element once, reusing it across the tile via an inner unrolled loop over the tile. The batch is zero-padded to a multiple of the tile size, and $\phi(\text{input})$ is precomputed on the host. This pushes the crossover with cuBLAS toward larger batches.

v3 — cached input activation. Because the input is clamped, $\phi(\text{input})$ is constant across all T steps, yet v1/v2 re-read it each step. v3 caches the $\phi(\text{input})$ tile once in shared memory (opting in to the larger per-block shared-memory budget on the target architecture), eliminating the roughly 256-fold redundant global reads of the input layer. This lifts the mid-batch speed further and moves the break-even point with cuBLAS to around batch 1024–1500.

The runtime dispatches by batch size: small batches take the per-sample path (v1), larger batches the tiled path (v2/v3). The activation is selected by an integer code (0 tanh, 1 identity, 2 sigmoid) matched between the Python wrapper and the device functions; the wrapper detects the activation from $\phi(0)$ and $\phi(1)$ and *raises* on anything unrecognised. This guard was added after a real bug in which sigmoid silently fell through to tanh because the detector keyed only on tanh’s signature — precisely the kind of silent-wrong-answer failure the project is built to avoid.

4.3.4 The general-depth variant

The v1/v2/v3 kernels hardcode the two-hidden-layer topology for speed, which was a documented limitation. We removed it with a general-depth kernel that settles a PCN of *arbitrary* depth L . The mechanism is to pass the weights, states, and biases not as fixed arguments but as arrays of device pointers (carried as 64-bit integer addresses), together with per-layer sizes and *dynamically computed* shared-memory offsets, so a single kernel can index any number of layers laid out contiguously in shared memory. The update is the identical Jacobi step used by the PyTorch reference. The dispatch keeps the optimised two-hidden-layer path for that common case and routes any other depth to the general kernel. It is per-sample only — the launch-bound small-batch regime the kernel targets — so a register-blocked tiled GEMM for large batch at large depth remains open. A platform pitfall worth recording: on Windows `long` is 32-bit, so the pointer arrays must be explicitly

64-bit integers; using the wrong type produces a size/type mismatch. The kernel is correctness-verified across depths two through five (one to four hidden layers) crossed with tanh and sigmoid and two batch sizes — sixteen configurations, all matching the PyTorch path to roughly 10^{-6} .

4.3.5 Correctness before speed

Following good benchmarking discipline, correctness was established before any timing: the kernel must reproduce the PyTorch reference, and only then is it timed. Verification uses `allclose` at roughly 10^{-6} against `_settle_pytorch` across activations (tanh, identity, sigmoid), clamp on and off, and both dispatch paths. One subtlety is load-bearing: the comparison must be run in a *stable* settling regime. An unstable configuration amplifies ordinary floating-point reordering differences between the two implementations into apparent “errors” that are really divergence of the dynamics, not a kernel bug. All timing measurements call `torch.cuda.synchronize()` before reading the clock, since CUDA execution is asynchronous and naive timing would measure launch latency rather than completion. The kernel is strictly optional: the default backend is PyTorch, fully functional, and the CUDA path is enabled only deliberately, by setting an environment flag and requesting `backend="cuda"`; without both, requesting the CUDA backend raises rather than silently falling back. The kernel speedups, the direct launch-count evidence, and the end-to-end integration are reported in Section 5.2; the launch-count and speedup figures are presented there.

4.4 The Fair-Comparison Protocol

The PC-vs-BP question is easy to answer badly. A casual comparison varies the learning rule *and* a dozen incidental factors at once, and then attributes the difference to the learning rule. Our protocol exists to isolate the one variable of interest, following the fairness discipline of Song et al. (2024).

4.4.1 Identical architecture, initialisation, and forward function

The backprop baseline, `BPMLPRef`, is constructed by *building a PCN at the same seed and cloning its weights and biases* into trainable parameters. This guarantees, by construction rather than by careful matching, that PC and BP start from bit-identical initial weights. The baseline’s forward pass reuses the *same* layer chain as

the PCN’s feedforward initialisation ($h \leftarrow \phi(h) W_i^\top + b_i$), so the two arms compute the identical function of their parameters; only the learning rule differs. Both arms draw from the same MNIST pipeline. The backprop arm *does* use autograd — that is the entire point, it is the backprop learner — and returns the same metrics dictionary schema as `train_and_eval`, so the search loop and comparison harness treat the two arms uniformly.

4.4.2 Matched loss — the dominant confound

The single most important control is the loss function. PC clamps the output to a one-hot target, which makes its implicit objective a squared error to that one-hot vector. A backprop baseline trained with cross-entropy is therefore *not* a like-for-like control: it differs from PC in both the learning rule and the objective. The protocol provides both BP arms: BP(MSE), which minimises the same squared error to the one-hot target that PC implicitly does and is the fair comparison, and BP(CE), the practical cross-entropy baseline reported for reference. The difference BP(CE) – BP(MSE) then isolates the pure loss effect. This matters because, as Section 5.3 reports, the loss function turns out to move the comparison far more than the learning rule does; an early, large apparent BP advantage was almost entirely the loss, not the rule.

4.4.3 Joint learning-rate tuning and the stability frontier

A second confound is asymmetric tuning. Sweeping only the weight learning rate η_w while fixing the state learning rate η_x systematically penalises PC, because the two interact through a stability frontier: at a given η_x , increasing η_w past a point causes the settling dynamics to diverge, collapsing accuracy to chance; lowering η_x restores stability and lets the same η_w succeed. The protocol therefore tunes η_x and η_w *jointly* for PC over a grid, and tunes each method’s learning rate independently. A related guard, learned through a false-positive, is that the backprop baseline must itself actually be *learning* before any comparison is meaningful: an untuned BP(MSE) that sits at chance can manufacture a spurious PC “win” that a learn-accuracy diagnostic immediately exposes.

4.4.4 Statistics: validation selection and bootstrap intervals

Learning rates are selected on a held-out validation split and reported on the test set, never selected on the test set. Every headline number is reported as a mean over at least three seeds with a bootstrap confidence interval, so that “A beats B”

is a statement about non-overlapping intervals rather than single-run noise. We are explicit about the interval width used: following Song, the bootstrap routine reports 68% (one-sigma) intervals, and we label them as such rather than implying 95%. The harness that implements bootstrap intervals and grid sweeps lives in the experiment protocol module and feeds the comparisons of Section 5.3.

4.5 Continual-Learning Setups

Continual learning is where PC’s prospective-configuration advantage is most often claimed, so it receives the most careful protocol design. All setups share the fair-comparison machinery above — identical architecture and initialisation, matched loss, per-method tuning — and add the canonical continual-learning metrics. We implement these in-house rather than via Avalanche (Lomonaco et al., 2021) to avoid dependency conflicts and keep the project genuinely from-scratch.

4.5.1 Metrics: the R-matrix, ACC, BWT, and learn-accuracy

After training task i we evaluate on every task j , filling the accuracy matrix $R[i][j]$ in the sense of Lopez-Paz and Ranzato (2017). From it we read the average final accuracy $\text{ACC} = \text{mean}_j R[T-1][j]$ and the backward transfer $\text{BWT} = \text{mean}_{j < T-1} (R[T-1][j] - R[j][j])$, where a negative BWT is forgetting. Critically, we also report *learn-accuracy*, the mean of the diagonal $R[j][j]$ — the accuracy on each task right after it is learned. This third metric exists to defeat the stability-plasticity confound: “PC forgets less” is only meaningful if PC also *learned* each task comparably well. A method that learns each task poorly will trivially appear to forget little, and learn-accuracy makes that visible.

4.5.2 Domain-incremental: Permuted-MNIST

Permuted-MNIST is domain-incremental: a fixed ten-way head throughout, with each task applying a fixed random pixel permutation. Because the head and architecture never change, PC and BP run through an identical task stream with no head surgery. Task zero is the identity permutation (plain MNIST). The PC arm trains each task with the standard settle-then-update epoch; the BP arm with SGD on its chosen loss; both are evaluated on every task after each, capping the per-task test set because a PC evaluation is itself a settle and the full quadratic evaluation grid would be costly.

4.5.3 Class-incremental: Split-MNIST and Split-FashionMNIST

The class-incremental setups partition the labels into disjoint class sets that *share* a single K -output head, with labels remapped to $0 \dots K - 1$ so the shared head is forced to overwrite — this interference is exactly the regime in which a PC advantage is claimed, unlike the milder domain-incremental case. Split-FashionMNIST (two disjoint five-class tasks) reproduces the shape of Song’s setup; Split-MNIST (five disjoint two-class tasks) is the other standard variant. Both (Xiao et al., 2017; LeCun et al., 1998) run through the same harness.

4.5.4 The Song-exact alternating protocol

The setups above train each task to completion before moving on. Song’s Figure 4 protocol is different and more demanding: two disjoint five-class FashionMNIST tasks share one five-output head and are trained by *minibatch-level alternation* — a few minibatch updates on the current task, then a switch, repeated. We implemented this faithfully: a network of four layers with thirty-two hidden units per layer, sigmoid activation, Xavier-normal initialisation, batch size thirty-two, switching every four updates. On the axes that define the protocol — four layers, thirty-two hidden, sigmoid, Xavier, two disjoint five-class tasks, shared head, alternation — this matches Song exactly, in contrast to our earlier sequential proxy.

Two methodological points are essential to report honestly here, both surfaced by an adversarial three-lens review (literature fidelity, statistics, and alternative explanations) that corrected a premature reading of the protocol. First, a framing correction: Song’s Figure 4e is *not* a “PC is robust across learning rates” claim — that is a different figure. Figure 4e shows *less catastrophic interference*, plotting the mean test error of both tasks against learning rate *with the learning rate tuned separately per method*. We therefore test interference, not learning-rate robustness, and we tune the learning rate per method. Second, a faithfulness-versus-trainability tension: Song’s exact budget of roughly eighty-four minibatch iterations leaves our network at chance. Rather than silently enlarge the budget, we make the training budget an explicit, controlled axis, sweeping from the exact eighty-four up to convergence. The reporting metrics distinguish the mean of both tasks’ final accuracy (Song’s Figure 4e metric) from the *minimum* of the two — the worse task being the true interference indicator, since a high mean with a low minimum is a sacrificed task, not balanced learning — alongside learn-accuracy and retention. This protocol is also where the CUDA kernel earns its keep as a real engine: the small batch size of thirty-two is exactly the launch-bound regime the kernel targets, and the alternating

study can be run on `backend="cuda"` with per-seed identical results (Section 5.4).

4.6 Generative PC

A distinctive property of predictive coding is that one mechanism — settling — can classify, generate, inpaint, and detect anomalies, depending only on what is clamped. We implement this without a new learning algorithm, by reusing the same PCN, settling, and weight update.

4.6.1 Orientation flip and generative training

The discriminative network maps image to label, and is therefore not a generative model of images: settling “backward” from a clamped label finds no learned image manifold and produces noise, as we verify honestly in Section 5.5. The fix is to flip the orientation. A generative PCN maps *label to image*, with the topology $[10, h, h, 784]$. It is trained generatively — the label is clamped at the input node, the image at the output node, the hidden states settle, and the same local Hebbian update is applied at equilibrium. Generation is then a forward pass from a clamped label; inpainting and anomaly detection are settling with different clamps. Crucially, no code in the settling or learning core changes: only the clamping pattern and the layer orientation differ.

4.6.2 Masked and input-free settling

The settling routine supports clamping the input fully (discriminative), leaving it free (generation), or masking it so that some pixels are clamped and the rest are free (inpainting). When the input is free, its update follows the same energy gradient, $-\phi'(s_0) \odot (W_0^\top \varepsilon_1)$, and a binary mask keeps the known pixels fixed while relaxing the unknown ones. The same per-sample residual energy $E = \frac{1}{2} \sum_k \|\varepsilon_k\|^2$, evaluated at the predicted label, serves as an anomaly score: an input the model cannot generate leaves high residual energy. This is the same quantity that drives settling, repurposed as a detector.

4.6.3 The output-dimension stability scaling

A reusable stability finding emerged from the orientation flip and is worth recording as method. With a 784-dimensional output, the hidden-layer settling gradient (which contains the term $\varepsilon_{\text{out}} W$) is roughly $\sqrt{784} \approx 28$ times larger than with a

ten-dimensional label output. A state learning rate that is well-behaved for the discriminative model therefore diverges to NaN for the generative one. The state learning rate must be scaled down by roughly the square root of the output dimension to remain stable. This is a general consequence of how the output error enters the hidden-state gradient, not a tuning accident, and it generalises the choice of η_x to other output sizes.

4.7 Optional Extensions: Incremental PC and EWC

Two further methods are implemented and validated, not merely sketched, to position vanilla SO-PC against close relatives.

4.7.1 Incremental PC

Incremental PC (Salvatori et al., 2024) differs from standard SO-PC in *when* the weights move. Rather than settling fully and updating once at equilibrium, it updates the states *and* the weights after every inference step, from the same error vector. The update is simultaneous: the state step uses the current weights and the weight step uses the pre-step states, both derived from one error vector, so neither sees the other’s change within a step. A clean correctness anchor pins this down — with the weight learning rate set to zero, one incremental step reduces *exactly* to one standard settling step, because incremental PC’s state update is the SO update; it merely also moves the weights.

The consequential property is sensitivity to the weight learning rate. Incremental PC applies T weight updates per batch rather than one, so its effective weight learning rate is roughly T times larger; at the standard η_w it diverges to NaN, and it needs a correspondingly smaller rate (on the order of η_w/T) to be stable. The incremental path is restricted to the PyTorch backend by design: the fused CUDA kernel settles *without* touching the weights, so it cannot express an update that writes the weights every step. The lesson — incremental PC needs a $\sim T$ -times smaller weight learning rate, after which it is competitive or better — is reported quantitatively in Section 5.6.

4.7.2 Elastic Weight Consolidation

To locate the absolute continual-learning numbers against a dedicated method, we add Elastic Weight Consolidation (Kirkpatrick et al., 2017) as a third class-

incremental arm on the backprop side. After each task we store the diagonal empirical Fisher information (the mean squared gradient of the task loss at the converged parameters) and the optimal parameters, and later tasks carry the quadratic anchor $\frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2$ that penalises moving parameters the earlier task found important. EWC necessarily uses autograd (it is a backprop-side baseline, not PC) and slots into the existing class-incremental harness as a method choice. A caveat carried over from the fair-comparison work: in a regime too weak for the base learner to actually learn, EWC is correctly inert, because there is nothing to forget. Its measured effect appears in Section 5.6.

4.8 Reproducibility and Tooling

Reproducibility is treated as part of the method, not an afterthought.

Environment. The project uses `uv` exclusively for environment management — never `pip` — so the dependency set is locked and reconstructable with a single command. The CUDA build is pinned to a toolkit-matched PyTorch (`cu126`) and requires `nvcc` with an MSVC toolchain on Windows; the kernel is compiled from the real `.cu` source via JIT loading and cached. The pure-PyTorch path requires no compilation at all, so the entire study — minus the optional kernel — runs with only `torch` and `torchvision` installed.

Offline test suite. A suite of forty tests validates the settling mathematics, the local update, the energy formula, the generative and masked-settling paths, and the harness wiring. The tests are hermetic and offline: they use small synthetic tensors with no MNIST download and run CPU-only, so correctness can be re-verified anywhere without a GPU or network access. Earlier weak assertions (for example, a training-loop test that only compared the last epoch to the first rather than checking the trend) were tightened in the course of the project.

Autonomous hyperparameter search. The Phase-4 search is a thin layer over `train_and_eval`: grid and random search (with an optional Optuna backend) propose a config, call the interface, and log the metrics, with no rewrite of the core. The single stable signature is what makes the search “almost free” in engineering terms. As a sanity check, an autonomous search run rediscovers the $\eta_x \leftrightarrow \eta_w$ stability interaction described in Section 4.4 — a useful confirmation that the protocol’s central confound is real and findable without prior knowledge.

Figure and artefact pipeline. Every quantitative claim is backed by a stored artefact: benchmark JSON for the kernel timings and launch counts, per-seed result files for the alternating sweep, and the figures of Chapter 5. Per-seed values are retained (not just aggregates) so that the spread behind every interval can be inspected, which is essential given that several of the study’s headline findings are null and rest on the *absence* of a separation rather than its presence.

Chapter 5

Experiments and Results

This chapter reports the empirical core of the thesis. It is deliberately organised as a *data* chapter: it states the experimental protocol, then presents measured numbers with their uncertainty (confidence intervals, seed counts) and the caveats that bound each measurement. The methods themselves — the State-Optimization (SO) settling rule, the local Hebbian weight update, the fused CUDA kernel design, and the fair-comparison protocol — were developed in Chapter 4; we reference them here rather than re-deriving them. Deeper interpretation of *why* the results look as they do, and what they imply for the field’s inconsistent claims about Predictive Coding (PC), is deferred to Chapter 6; the boundaries of what the numbers license are collected in Chapter 7.

Two stylistic commitments govern the entire chapter, and they are the project’s defining virtue. First, *the headline findings are largely null*: under a fair comparison, vanilla SO-PC performs essentially on par with backpropagation (BP) at matched loss, and the one genuine “better” result is an engineering win (the kernel) bounded by an honest crossover. Second, *every claim is reported with its statistical reach*. Where an effect is directional but inside the seed-to-seed spread, we say so explicitly and decline to call it significant. A sober reckoning is the contribution; inflating a marginal edge would undermine it.

5.1 Experimental Setup

5.1.1 Hardware, software, and reproducibility

All GPU experiments were run on a single NVIDIA GeForce RTX 3080 Ti (12 GB, compute capability `sm_86`) under PyTorch 2.12.0+cu126 with the CUDA 12.6 toolkit. PyTorch is used *only* as a tensor engine and, for the BP baselines, as

an autograd engine; the PC learning rule itself uses no autograd — every state and weight update is computed by hand under `@torch.no_grad()` (see Chapter 4). The hand-derived settling and Hebbian gradients were checked against `torch.autograd` to $\sim 10^{-5}$ in the test suite, where autograd serves purely as an external oracle. A small set of early correctness and micro-benchmark runs were performed on CPU (`torch 2.12.0+cpu`); these are flagged where they appear and are not used for any headline claim.

The complete codebase, the test suite, the figure-generation script, and the raw experiment artifacts (the `results/*.json` files) are released so that every number in this chapter can be regenerated. The four figures are produced by `scripts/make_figures.py` directly from those JSON artifacts, so the plotted values and the tabulated values share a single source of truth. Reproduction details (commands, environment, seeds) are collected in Appendix A.4, and the per-seed values underlying the aggregate statistics are stored in the artifacts (e.g. `results/m4_alternating_song_exact_budgetsweep_s10.json`).

5.1.2 Datasets and architecture

The discriminative experiments use MNIST (LeCun et al., 1998); the continual-learning and Song-faithful experiments use FashionMNIST (Xiao et al., 2017) and class-split variants of both. Unless stated otherwise the discriminative network is the multilayer perceptron (MLP) topology $[784, 256, 256, 10]$: a 784-dimensional input layer, two hidden layers of 256 units, and a 10-way output. The activation ϕ is tanh for the discriminative runs. The Song-faithful replication (Section 5.4) instead uses Song’s architecture exactly: a four-layer $[784, 32, 32, 5]$ network with *sigmoid* activations and Xavier-normal initialisation. The generative model (Section 5.5) reverses the orientation to $[10, 256, 256, 784]$ (label→image).

5.1.3 Metrics

We report, depending on the experiment: test accuracy; the free-energy $E = \frac{1}{2} \sum_k \|\varepsilon_k\|^2$ and its convergence behaviour; `settling_steps_to_converge` (the number of inference steps until the relative energy-delta falls below a tolerance `tol`, which turns “did the network settle?” into a measurable quantity); noise robustness (test accuracy under additive Gaussian input noise of standard deviation σ); wall-clock milliseconds per settle and per settling step; the profiled CUDA-launch count per settle; and, for continual learning, the standard GEM-style metrics (Lopez-Paz and Ranzato, 2017) — final average accuracy, learning accuracy (the diagonal of the

task×task accuracy matrix, i.e. how well each task was learned *when* it was the current task), backward transfer (BWT, the average change in earlier tasks’ accuracy after later training), and retained task-0 accuracy. For the generative model we report the area under the ROC curve (AUC) for out-of-distribution (OOD) detection by residual energy. The continual-learning harness is implemented leanly without the `avalanche` library (Lomonaco et al., 2021), against which the design was sanity-checked.

5.1.4 Statistics

Every aggregate number is reported as a mean over multiple random seeds together with a bootstrap confidence interval (CI). **We follow Song et al. in reporting 68 % (one- σ) intervals, *not* 95 %;** the bootstrap routine (`protocol.py`) computes the 1σ band deliberately. This matters for honest reading: a one- σ interval is narrower than a 95 % interval, so the standard for “the CIs overlap, hence indistinguishable” is correspondingly conservative — if two methods’ 1σ bands overlap they are certainly not separated at 95 %. The fair PC-vs-BP study (Section 5.3) uses three seeds; the Song-faithful budget sweep (Section 5.4) was run at five seeds and then re-run at ten. Where we describe an effect as “directional but not significant” we mean specifically that the point estimate has a sign but the gap is smaller than the (often large) seed-to-seed spread.

5.2 The Fused CUDA Settling Kernel

This section reports the project’s one genuine “faster” result, an engineering win whose value lies as much in the measurement as in the speedup. The kernel design is described in Section 4.3; here we measure what it buys, where it stops buying it, and why.

5.2.1 Correctness comes first

Before any timing, the kernel was held to a correctness gate. The fused CUDA kernel reproduces the PyTorch settling backend (`_settle_pytorch`) to within $\sim 10^{-6}$ (`allclose`) across activations `{tanh, identity, sigmoid}` and with input clamping on and off (`scripts/verify_kernel.py`). One subtlety is worth recording because it is a recurring trap: the equivalence must be checked in a *stable* settling regime. If the state learning rate η_x is too large the iteration is numerically unstable, and unstable settling amplifies the inevitable floating-point reordering between a fused kernel and

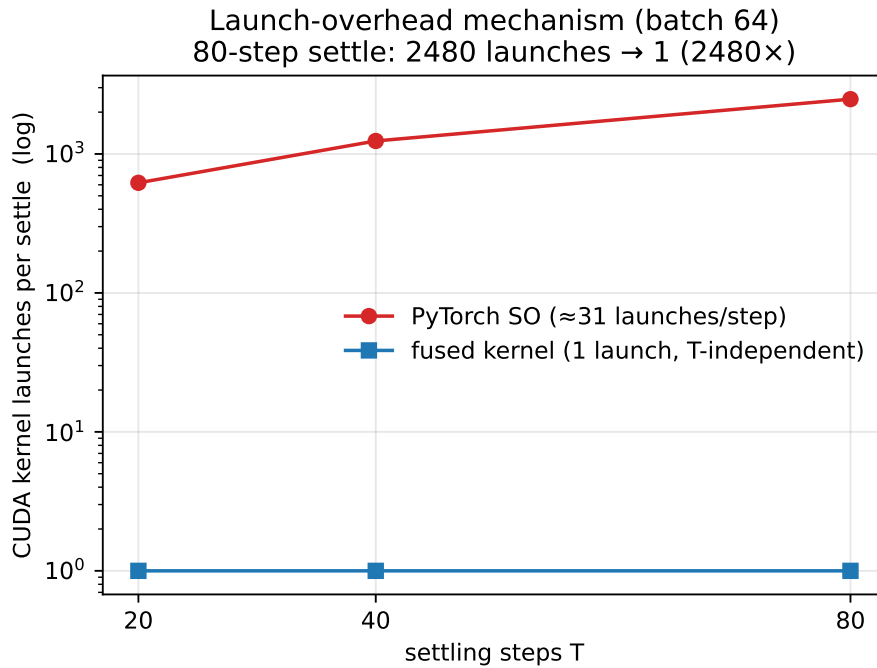


Figure 5.1: CUDA kernel launches per full settle versus the number of settling steps T (batch 64, RTX 3080 Ti). The PyTorch SO backend issues 31 launches per step ($620 \rightarrow 1240 \rightarrow 2480$ for $T = 20, 40, 80$); the fused kernel issues exactly one, independent of T . This is the mechanistic confirmation that PC inference at this scale is launch-overhead-bound rather than compute-bound.

a sequence of cuBLAS calls — the two backends then diverge not because either is wrong but because the dynamics are ill-conditioned. Verifying under a stable configuration isolates correctness from conditioning.

5.2.2 PC inference is launch-overhead-bound

The central measurement is mechanistic, not merely a wall-clock comparison. Using the PyTorch profiler to count `cudaLaunchKernel` events (`scripts/profile_launches.py`; `nsys` was unavailable, so we count launch events directly), we find that the PyTorch SO backend issues **31 kernel launches per settling step**, and therefore 620, 1240, and 2480 launches for a full settle of $T = 20, 40$, and 80 steps respectively. The fused kernel issues **exactly one launch**, independent of T — a reduction of 620–2480 $\times$ (Figure 5.1). This is the *mechanistic* evidence that launch overhead, not arithmetic, is the bottleneck the kernel removes: it answers the solver-centric objection (one might suspect the speedup is an artifact of an inefficient PyTorch graph) by showing the launch count directly rather than inferring it from timing.

Consistent with a launch-bound regime, the PyTorch SO settle costs roughly 0.5 ms per step *independent of batch size* from $B = 64$ up to $B = 4096$ (the per-

step wall clock barely moves while the work per step grows $64\times$), and the GPU is only $\sim 25\text{--}49\%$ utilised during the M4 runs. We note that this “framework tax” of many tiny operator launches is a generic, previously documented phenomenon, not a discovery of this thesis; the contribution is its quantification for PC inference and the kernel that removes it.

5.2.3 Speedup and the architectural crossover

Figure 5.2 plots the speedup of the fused kernel over the PyTorch backend versus batch size for the topology $[784, 256, 256, 10]$. The kernel wins decisively in the small-batch regime, reaches break-even in the middle, and loses at large batch. With the final batch-tiling and input-activation caching optimisations (v3), the measured behaviour is: **about $3.2\times$ at $B = 64$, about $2.0\times$ at $B = 256$, break-even ($\sim 1.0\times$) around $B = 1024$, and a loss ($\sim 0.33\times$) at $B \geq 2048$** . The optimisation history is itself informative: the naive v1 kernel (one block per sample, each weight re-read from global memory per sample) already gave $3.2\times$ at $B = 64$ but lost $3\times$ at $B = 1024$ because it is bandwidth-limited; v2 processes a tile of 8 samples per block and reuses each weight across the tile (the key GEMM optimisation); v3 additionally caches the clamped — and therefore constant — $\phi(\text{input})$ tile in shared memory, removing $\sim 256\times$ redundant global reads of the first weight block. These optimisations moved the crossover from ~ 256 to $\sim 1024\text{--}1500$.

The large-batch loss is not a tuning failure but a fundamental architectural trade-off, and we report it as a positive systems finding rather than as “not yet optimised”. At large batch the workload becomes *compute*-bound: the naive in-kernel GEMM uses scalar fused multiply-adds with no register blocking and reaches only a fraction of the FLOP peak, whereas cuBLAS is register-blocked and runs near optimum. Crucially, **the fused-resident design and a cuBLAS-class GEMM are architecturally opposed**: keeping all T states resident in registers and shared memory (which is what kills the launch overhead) consumes precisely the register and shared-memory budget that a fast tiled GEMM would need. The two goals compete for the same scarce resource. The honest conclusion is therefore that the fused kernel is the right tool in the small-to-medium batch regime ($B \lesssim 1024$, $1\text{--}3\times$, the realistic MNIST-scale PC training regime), and the PyTorch/cuBLAS backend is the right tool at $B \geq 2048$; a fully register-blocked tiled in-kernel GEMM would amount to re-implementing cuBLAS and is out of scope.

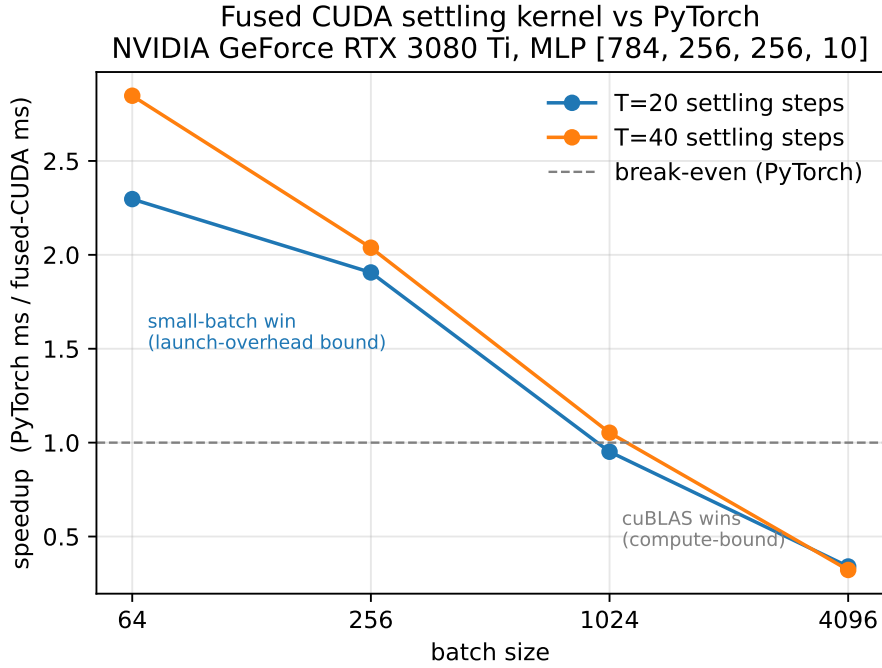


Figure 5.2: Fused CUDA settling-kernel speedup over the PyTorch backend versus batch size (RTX 3080 Ti, MLP [784, 256, 256, 10], settling steps $T = 20$ and 40). The kernel wins at small batch (launch-overhead bound, $\sim 3.2\times$ at $B = 64$, $\sim 2.0\times$ at $B = 256$), reaches break-even around $B \approx 1024$, and is outperformed by cuBLAS at $B \geq 2048$, where the workload becomes compute-bound. The small/medium-batch regime is the one relevant to online and continual PC training.

5.2.4 Generality and in-study use

Two facts elevate the kernel from a microbenchmark to a usable engine. First, it is no longer tied to a fixed topology. The original kernel was specialised to two hidden layers (`model.n==3`); a *general-depth* per-sample variant (`pcn_settle_so_deep`) passes the per-layer weights, states, and biases as arrays of device pointers (as `int64` addresses) with shared-memory offsets computed at launch, and runs the same Jacobi update as the PyTorch backend for arbitrary depth. It was validated to $\sim 10^{-6}$ (`allclose`) across depths 2–5 (one to four hidden layers) $\times \{\tanh, \text{sigmoid}\} \times B \in \{32, 256\}$ — **all 16 configurations** matching the PyTorch equilibrium. (A Windows portability trap was fixed in the process: `long` is 32-bit on MSVC, so the pointer arrays must be typed `int64_t`.) Second, the kernel was used as the *back-end of the project’s own Song-faithful study* (Section 5.4), which runs at batch 32 — exactly the small-batch regime the kernel targets. On that real workload it delivers $1.45\times$ **end-to-end** wall-clock (103.9s $\rightarrow$ 71.6s at budget 800, ten seeds) with **per-seed bit-identical results** (maximum accuracy difference 0.00 pp). The modest $1.45\times$ — rather than the $3.2\times$ of the $B = 64$ microbenchmark — is honest: the

training loop gains in the small-batch regime, but evaluation settles at batch 512, which dilutes the average. Integrating the kernel into the study also exposed and fixed a silent activation bug (`sigmoid` had been mapped to `tanh` internally), now guarded.

5.2.5 Novelty scope

To our knowledge this is the first hand-written, fused CUDA settling kernel *for PC*. The claim is bounded by a direct audit of eleven open PC libraries (none ship a `.cu` file; all are JAX/JIT — e.g. PCX (Pinchetti et al., 2024), JPC (Millidge et al., 2024) — or PyTorch-autograd, e.g. PRECO (van Zwol et al., 2024)) and, for double assurance, of the closest algorithmic relative, the Equilibrium Propagation family (Scellier and Bengio, 2017; Laborieux et al., 2021), which likewise ships no hand-written CUDA settling kernel (Chapter 3; Appendix A.2). We are careful *not* to claim the persistent, states-resident, single-launch *technique* as novel — it is established prior art (Baidu/cuDNN persistent RNNs (Diamos et al., 2016), recent LLM megakernels). The contribution is the transfer of that technique to PC, plus the measurement.

5.3 Fair PC versus BP

This is the methodological heart of the thesis. The protocol (Section 4.4) holds the architecture, the initialisation, the data, and — critically — the loss function fixed, so that the *only* variable is the learning rule; the learning rate is tuned per method on a held-out validation split and *reported* on test; every number is a mean over three seeds with a 68% bootstrap CI. Under this protocol the headline result is null: PC is statistically indistinguishable from loss-matched BP.

5.3.1 A methodological prerequisite: tune η_x and η_w jointly

A non-obvious lesson preceded the fair numbers and is reportable in its own right. An early comparison fixed the state learning rate at $\eta_x = 0.1$ and swept only the weight learning rate η_w ; PC then appeared to lose by ~ 10 pp. A two-dimensional $\eta_x \times \eta_w$ scan revealed a *stability frontier* coupling the two: at $\eta_x = 0.1$, PC *diverges* for $\eta_w = 0.05$ (collapsing to chance, 9.8%), whereas at the lower $\eta_x = 0.05$ the same $\eta_w = 0.05$ is stable and reaches 84.9%. With joint tuning ($\eta_x = 0.05$, $\eta_w = 0.05$) the apparent accuracy gap shrinks from ~ 10 to ~ 4 pp. **Sweeping only η_w**

Table 5.1: Deconfounded bulk comparison on MNIST (10k examples, matched MSE loss unless noted; mean over 3 seeds with 68% bootstrap CI). The fair PC-vs-BP(MSE) gap is +0.4 pp with overlapping CIs; the loss effect BP(CE) – BP(MSE) is +5.9 pp.

Arm	Test accuracy	Noise $\sigma = 1.0$
PC (MSE)	83.5% [82.8, 84.3]	52.6%
BP (MSE) — fair arm	83.9% [83.6, 84.3]	60.4%
BP (CE) — reference	89.9% [89.7, 90.0]	81.0%

systematically handicaps PC; the ~ 10 pp was largely a tuning artifact. The state and weight learning rates must be tuned together.

5.3.2 The confounded first results and their correction

An adversarial methodology review (four critic agents) found that the project’s initial “PC wins” were artifacts. Two confounds were identified at this stage and a third in the class-incremental setting below.

- **Confound 1 — loss mismatch.** PC minimises MSE to a one-hot target, but the first BP baseline minimised cross-entropy (CE). That violates “only the learning rule varies”. The fix is to add a **BP(MSE)** control arm with the *identical* objective to PC. The fair comparison is **PC(MSE) vs BP(MSE)**; the quantity BP(CE) – BP(MSE) isolates the pure loss effect.
- **Confound 2 — the plasticity confound.** In continual learning, “forgets less” can simply mean “learns each task less well”. The fix is to additionally measure learning accuracy (the diagonal of the accuracy matrix) and retained task-0 accuracy, so that retention is read against how much was actually learned.

5.3.3 Bulk accuracy and noise robustness

Figure 5.3 shows the deconfounded comparison; the bulk numbers (10k training examples, 8 epochs, $T = 40$, jointly tuned, three seeds) are:

The **fair gap PC(MSE) vs BP(MSE) is +0.40 pp, and the CIs overlap** — the two learning rules are statistically indistinguishable. The **loss effect BP(CE) – BP(MSE) is +5.92 pp**. In other words, the naive “BP beats PC by 4–10 pp” that the literature might lead one to expect is here *almost entirely the loss function, not the learning rule*. On noise robustness the story is similar in shape: BP(MSE)

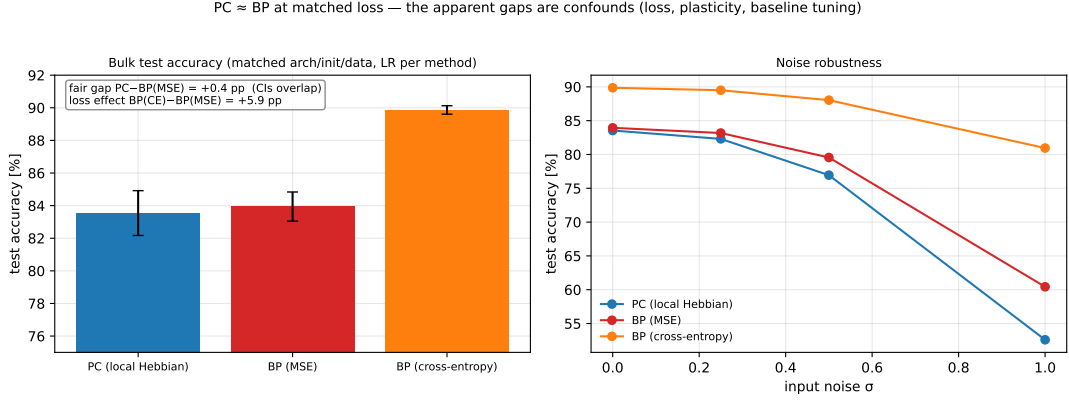


Figure 5.3: Matched-loss PC versus BP on MNIST (3 seeds, 68% bootstrap CIs). *Left*: bulk test accuracy — PC(MSE) and BP(MSE) are indistinguishable (CIs overlap); BP(CE) leads by the +5.9 pp loss effect, not by the learning rule. *Right*: test accuracy versus additive input-noise σ — BP(MSE) is modestly more robust than PC(MSE), but the large naive gap is again mostly the loss function.

Table 5.2: Permuted-MNIST domain-IL (5 tasks, matched loss; mean over 3 seeds, 68% CI on retained task-0). Against fair BP(MSE), BP forgets *less* and learns each task better; the earlier “PC forgets less” was an artifact of comparing to BP(CE).

Arm	Final ACC	Learn ACC	BWT	Retained task-0
PC (MSE)	55.0%	65.1%	−12.7%	41.8% [39.8, 43.8]
BP (MSE)	62.9%	69.1%	− 7.8%	54.2% [54.1, 54.3]
BP (CE)	60.7%	78.7%	−22.5%	47.2%

remains about 8 pp more robust than PC(MSE) at $\sigma = 1.0$ (60.4% vs 52.6%) — a real but small effect, dwarfed by the ~ 28 pp gap one would have wrongly attributed to PC’s disadvantage had the comparison been against BP(CE). A separate ablation confirmed that the settling budget is not hiding anything: varying $T \in \{20, 40, 80\}$ does not change bulk accuracy, ruling out an under-settling artifact.

5.3.4 Continual learning: domain-incremental

On Permuted-MNIST (five tasks, domain-incremental), against the fair BP(MSE) arm, the earlier “PC forgets less” headline collapses:

The headline was *doubly* artifactual. (1) It compared against BP(CE), which forgets more because of aggressive softmax updates; against the fair BP(MSE), BP forgets *less* (−7.8% vs −12.7%). (2) PC learns each task less well (learning accuracy 65.1% vs 69.1%) and retains task-0 *absolutely* worse (41.8% vs 54.2%, disjoint CIs) — the plasticity confound. The decision is therefore *not proven*.

Table 5.3: Class-IL parity (mean over 3 seeds, 68% CI on retained task-0). Against a correctly tuned, *learning* BP(MSE) baseline, PC shows no advantage in either split.

Setup	Arm	Final ACC	Learn ACC	BWT	Retained T0	
Split-FashionMNIST (2×5)	PC	60.9%	84.3%	−46.8%	37.8%	[34.3, 41.3]
	BP(MSE)	61.3%	86.1%	−49.6%	37.3%	[34.0, 40.6]
Split-MNIST (5×2)	PC	70.8%	97.4%	−33.3%	60.1%	[58.6, 61.7]
	BP(MSE)	73.4%	97.5%	−30.2%	62.0%	[60.4, 63.7] NO

5.3.5 Continual learning: class-incremental (Song’s regime)

The class-incremental setting (Split-FashionMNIST, two tasks of five classes; Split-MNIST, five tasks of two classes), with the BP learning rate tuned per arm by learning accuracy, is where a third confound surfaced:

Even in Song’s own class-IL interference regime, against a correctly tuned, *learning* BP(MSE), there is **no PC advantage**: PC and BP(MSE) are statistically indistinguishable on Split-FashionMNIST, and BP is marginally better on Split-MNIST. The third confound is the **untuned-baseline confound**: an untuned BP(MSE) ($\eta_w = 0.1$) stalls at chance ($\sim 50\%$) on Split-MNIST, manufacturing a false PC “win” that the learning-accuracy diagnostic caught. What remains is a consistent stability–plasticity trade-off — BP(CE) learns the most and forgets the most, PC learns the least and forgets the least — but PC sits *on the same curve* as loss-matched BP, not above it.

5.3.6 Synthesis: three confounds, one null

Across every tested regime (bulk accuracy, noise, sample efficiency, Permuted-MNIST domain-IL, Split-FashionMNIST and Split-MNIST class-IL), with matched loss and fairly tuned baselines, **vanilla SO-PC shows no advantage over BP** — on par in accuracy and learning, on the same stability–plasticity curve, slightly worse under noise. All three previously observed “PC wins” were methodological artifacts. The scientific value, as argued in Chapter 6, is a clean null result *plus* the systematic exposure of three confounds — the loss function, the plasticity confound in forgetting metrics, and an untuned baseline learning rate — that bias naive PC-vs-BP comparisons. We stress what this does *not* mean: it does not globally refute Song et al. (2024), whose setup differs in training schedule, activation, width, and possibly metrics. The defensible claim is bounded: *at MNIST/FashionMNIST scale with a fairly tuned BP baseline, the PC advantage disappears*. The strongest

test of that boundary — a faithful replication of Song’s exact protocol — is the subject of the next section.

5.4 Architecture-Faithful Replication of Song’s Exact Protocol

The class-IL results above follow Song’s *regime* but not his exact training. To probe whether the null survives a strict replication, we built the most architecture-faithful reproduction of Song et al. (2024) we could, and we use it to illustrate how an adversarial verification caught a premature finding before it reached the writeup.

5.4.1 What was built

The setup (`run_alternating`) pits two disjoint five-class tasks from FashionMNIST against each other, sharing a *single* five-output head (which forces interference), trained by **minibatch-level alternation** (`swap_every=4` updates, then switch task; batch 32). The network is [784, 32, 32, 5] with **sigmoid** activations and **Xavier-normal** initialisation. Checked against Song’s Methods section, this matches on the faithful axes: four layers, 32 hidden units per layer, sigmoid, Xavier, two disjoint five-class tasks, shared head, alternating training — in contrast to the Section 5.3 proxy, which was sequential, tanh, and [256, 256].

5.4.2 The framing correction (an adversarial three-lens review)

Three independent verification lenses — literature faithfulness, statistics, and alternative-explanations — dismantled the first draft of this experiment; all three corrections are folded in. The most important is the **framing correction** from the literature lens. Song’s Fig. 4e is *not* a “PC is robust across learning rates” claim (that is his Fig. 3, on target alignment). Fig. 4e shows *less catastrophic interference*, with the learning rate optimised *separately per method* (its caption plots the mean test error of both tasks versus learning rate). We had initially misattributed it as a learning-rate-robustness claim. The corrected experiment therefore tests *interference* (not LR robustness), tunes the learning rate per method, and reports the worse of the two tasks’ accuracies (*min-both*) as the true interference tell alongside Song’s mean-both metric. This misframing-then-correction is itself a reusable safeguard, discussed in Chapter 6.

Table 5.4: Song-exact alternating Split-FashionMNIST, ten seeds. Mean-both and worse-task min-both accuracy (%) with $\pm 1\sigma$ bootstrap intervals; $\Delta_{\min}$ is PC – BP(MSE). At *no* budget does the PC-vs-BP(MSE) gap exceed 1σ (the seed spread σ is ~ 4 –11 pp on each arm). The pattern is directional only.

Budget (iters)	PC mean/min	BP(MSE) mean/min	BP(CE) mean/min	$\Delta_{\min}$
84 (Song exact)	26.6 / 24.8	28.0 / 26.2	29.6 / 27.9	−1.5
250	39.2 / 36.2	38.4 / 34.7	41.0 / 36.3	+1.5
800	64.1 / 55.8	50.6 / 44.8	58.4 / 49.8	+11.0
2500 (converged)	73.6 / 68.9	68.5 / 63.2	74.3 / 69.6	+5.7

5.4.3 Budget as a controlled axis

A faithfulness-versus-trainability tension appeared: Song’s *exact* budget (~ 84 mini-batch iterations, swapping every 4) leaves *every* method at chance in our hands (all arms ~ 20 –30% at a chance level of 20%). Rather than silently enlarge the budget, we make it a **controlled axis**: 84 (exact) $\rightarrow$ 250 $\rightarrow$ 800 $\rightarrow$ 2500 (convergence), and report the whole curve.

5.4.4 Results

Figure 5.4 and Table 5.4 report the ten-seed sweep, run on the fused CUDA kernel (per-seed identical to PyTorch, which doubles as a practical test of the kernel integration). The metric *min-both* — the weaker of the two tasks — is the genuine interference indicator; *mean-both* is Song’s reported metric. The learning rate was tuned per method (it landed at 0.1 at the trainable budgets).

Verdict (calibrated): at $n = 10$ no PC-vs-BP(MSE) difference is separated by $\geq 1\sigma$ at any budget. The per-arm seed spread ($\sigma \approx 4$ –11 pp) is large relative to every point-estimate gap, so the effects are *directional, not significant*. The direction is consistent and worth recording: PC learns *slower early* (at budgets 84 and 250 it trails BP(MSE) by a few points — settling needs more steps to reach the “prospective configuration”), then *catches up and edges marginally ahead by convergence*. At the converged budget of 2500 the point estimates favour PC by +5.0 pp mean-both and +5.7 pp min-both; at budget 800 the point gap is even larger (+13.5 and +11.0 pp) but the spread there is enormous ($\sigma_{\text{sum}} \sim 9$ –16 pp), so any single-budget claim is noise. Notably the min-both edge equals or exceeds the mean-both edge, a *weak* hint that PC balances the two tasks slightly better — but it is inside the noise. BP(CE) leads throughout (the same loss confound), so it carries no information about the learning rule.

This is the *first* regime in which vanilla PC does not sit *behind* loss-matched BP

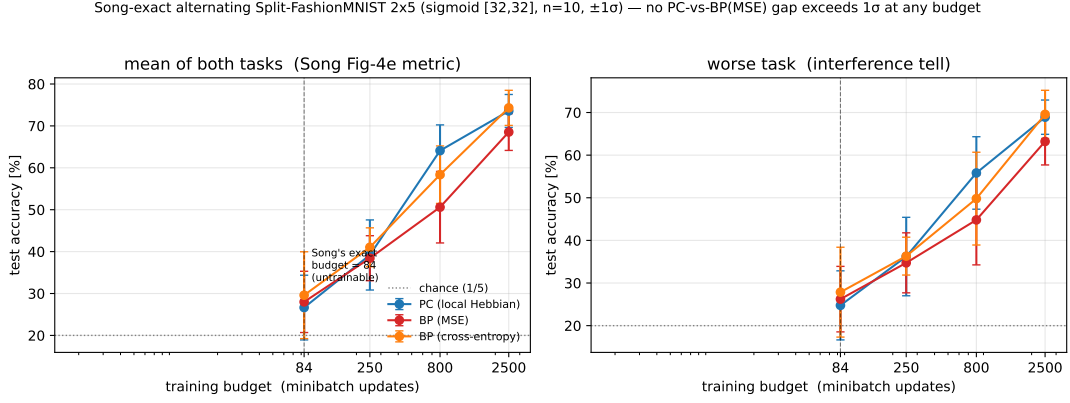


Figure 5.4: Song-exact alternating Split-FashionMNIST: mean-both (Song’s metric) and worse-task min-both (interference tell) accuracy versus training budget ($n = 10, \pm 1\sigma$). No PC-vs-BP(MSE) gap exceeds 1σ at any budget. PC learns slower early and pulls marginally ahead by convergence, but the gap stays inside the (large) seed spread; raising the seed count from five to ten did not make it significant. BP(CE) leads throughout — the same loss confound as Section 5.3.

at the convergence point (in Section 5.3 PC was on par to slightly behind). It neither refutes Song (different budget semantics and scale) nor confirms him (the effect is not separable from zero). The honest conclusion: the faithful replication reproduces no clear PC interference advantage at ten seeds, consistent with the overall “PC $\approx$ BP under fair tuning” verdict. We also record the disclosed deviations from Song: a fixed $T = 20$ (no convergence tolerance, so under-settling may co-explain PC’s early slowness), and the budget itself (Song’s 84 iterations are untrainable in our hands). The statistics lens additionally corrected a labelling slip in an earlier intermediate report: the intervals are 68% (one- σ) à la Song, not 95%.

5.4.5 Why the kernel matters here

This section is the regime *for which* the fused settling kernel was planned: PC settling at batch 32 is launch-overhead-bound (Section 5.2), exactly what the kernel removes. Run end-to-end on the real budget-800 workload, the kernel gives the $1.45\times$ reported above at per-seed-identical results. This is the concrete link between the systems contribution (Hook A) and the science contribution (Hook B): the hand-written kernel is the engine of a real study, not a benchmark artifact.

5.5 Generative Predictive Coding

This section demonstrates PC’s distinctive “one mechanism, many tasks” property — and, equally honestly, shows that it only appears when the model is *generatively*

Table 5.5: The same settling rule, two training orientations. Generative PC recovers the capabilities a discriminative PC cannot provide.

Capability	Discriminative (§5.5)	Generative
Generation (clamp label $\rightarrow$ image)	grey noise	recognisable per-class prototypes 0–9
Inpainting (fill occluded half)	noise fill	structured (blurry) reconstruction
Anomaly (MNIST vs uniform OOD)	AUC 0.44	AUC 1.00

trained.

5.5.1 A discriminative PC cannot generate

On the discriminatively trained network (test accuracy 90.6%), running the same settling machinery “backwards” fails by construction. Clamping a label and settling the input yields grey noise, not a recognisable digit; inpainting an occluded half preserves the visible pixels exactly but fills the occluded region with noise; and OOD detection by residual energy gives an AUC of only 0.438 against uniform-noise OOD — no separation. This is architectural, not a tuning failure. The discriminative PC clamps input *and* label and settles only the hidden states with a feedforward-predictive structure ($\text{pred}_{i+1} = W_i \phi(s_i) + b_i$); it learns an input $\rightarrow$ label map, not a generative model of the inputs. There is no learned image manifold for “backward” settling to find, and with the input clamped the free states can always relax to the zero-energy feedforward solution, so residual energy cannot separate OOD.

5.5.2 A generative PC delivers the full capability set

Reversing the orientation to [10, 256, 256, 784] (label $\rightarrow$ image) and training *generatively* (clamp the label at the input and the image at the output, settle the hidden states, apply the local Hebbian update — the same PCN, `settle`, and `weight_update`) yields a generative model. The results (30k images, 10 epochs, GPU; `scripts/demo_generative_v2.py`) are summarised in Table 5.5.

The generative model produces **recognisable per-class digit prototypes**, reconstructs occluded halves with structured (if blurry) content, and separates uniform-noise OOD with a perfect **anomaly AUC of 1.00** (in-distribution residual energy 5.1 versus OOD 94.5). The same settling mechanism, varying only what is clamped, classifies, generates, inpaints, and detects anomalies. A stability finding is reportable on its own: the 784-dimensional output makes the hidden-settling gradient ($\varepsilon_{\text{out}}@W$) about $\sqrt{784} \approx 28\times$ larger than for the 10-dimensional label head, so a state learning rate of 0.1 (fine for the discriminative model) diverges to NaN; the

state LR must scale as $\sim 1/\sqrt{\text{output-dim}}$ ($\eta_x \approx 0.01$) for stable settling.

The honest caveats temper the win. A one-hot input produces only ten distinct outputs, so these are *class prototypes* (the per-class mean), not diverse samples; diversity would require a continuous latent with a prior (VAE-style). Inpainting is prototype-driven, hence blurry rather than pixel-sharp. And uniform noise is an *easy* OOD, so AUC 1.0 is not a hard test; a near-OOD such as FashionMNIST or rotated digits would be the harder next step. The interpretive significance of the discriminative/generative distinction is taken up in Chapter 6.

5.6 Optional Extensions: iPC and EWC

Two further variants were implemented *and* validated, each tightening a loose end. They are optional depth rather than headline results.

5.6.1 Incremental PC (iPC)

Incremental PC (Salvatori et al., 2024) (`update_variant="ipc"`) interleaves a weight update with *every* settling step, rather than settling fully and updating once at equilibrium. A correctness anchor pins it to the standard rule: with $\eta_w = 0$, one iPC step reproduces a standard settling step *exactly* (iPC’s state update *is* the SO update; it merely also moves the weights). The empirical finding is that **iPC is markedly more learning-rate-sensitive**: because it applies T weight updates per minibatch, its effective weight LR is $\sim T\times$ larger, and at the standard $\eta_w = 0.02$ it diverges (NaN). At a $\sim T\times$ -smaller rate ($\eta_w \approx 0.005$, i.e. $\sim \eta_w/T$) it is stable and, in a fast-training regime (MNIST, 3k samples, 2 epochs), reaches 74.3% **versus** 63.1% **for standard PC** — iPC learns faster per step, consistent with Salvatori et al. (2024). The lesson: iPC trades learning-rate sensitivity for faster per-step learning, and needs a $\sim T\times$ -smaller weight LR to be competitive.

5.6.2 EWC as a continual-learning positive control

Elastic Weight Consolidation (Kirkpatrick et al., 2017) was added as a third class-IL arm (`run_class_il(method="ewc")`): after each task the diagonal empirical Fisher (mean squared gradients) and the optimal weights are stored, and later tasks carry the quadratic anchor $(\lambda/2) \sum_i F_i(\theta_i - \theta_i^*)^2$. Validated end-to-end on Split-MNIST (5×2 , in a *learning* regime), EWC **forgets less than the matched BP(MSE): BWT** -41.1% **vs** -45.7% , **final accuracy** 65.7% **vs** 62.0%, at equal learning (learning accuracy 98.5%). This is exactly the expected EWC effect and serves as

a *positive control*: it confirms the continual-learning harness measures forgetting correctly. (Consistent with the Section 5.3 lesson, in a too-weak regime — where BP is stuck at chance — EWC is inert, because there is nothing learned to forget.)

5.7 Autonomous Hyperparameter Search

The final experiment is an internal consistency check rather than a discovery. A bounded random search over (T, η_x, η_w) , implemented as a thin layer over the `train_and_eval` interface (Chapter 4), independently rediscovers the $\eta_x \leftrightarrow \eta_w$ interaction surfaced manually in Section 5.3. The best configuration in the run prefers a *low* state learning rate, a *high* weight learning rate, and a *high* settling budget ($\eta_x = 0.02$, $\eta_w = 0.05$, $T = 40$, reaching 77.6% on the short MNIST run in `results/m7_search.json`), where the high η_w is only stable *because* η_x is low — precisely the stability frontier identified by hand. The search also records `settling_steps_to_converge` per configuration (e.g. 94 steps at the high- η_w /low- η_x optimum versus ~ 30 at the more aggressive settings), confirming that the better-learning regime is also the slower-settling one. That an automated search re-derives a manually found coupling is a small but reassuring validation that the hyperparameter landscape was characterised correctly, and that the `train_and_eval` contract is sound.

5.8 Depth, Signal Decay, and a Local Precision Mitigation

The parity results above use shallow networks. Predictive coding is known to degrade with depth, and we close the experimental chapter by reproducing this phenomenon at our scale and probing one local mitigation — both to characterise where vanilla SO-PC breaks and to be explicit that the remedy is *not* novel.

5.8.1 Reproducing the signal decay

Standard SO-PC matches BP at one or two hidden layers but collapses as depth grows, while the BP baseline (same initialisation) is essentially flat (Table 5.6). The mechanism is the exponential signal decay of Section 2.3: at the deepest network the per-layer learning signal $\|\Delta W_i\|$ falls by a factor of ~ 669 from the output-side to the input-side layer, so early layers receive almost no signal and never train. This

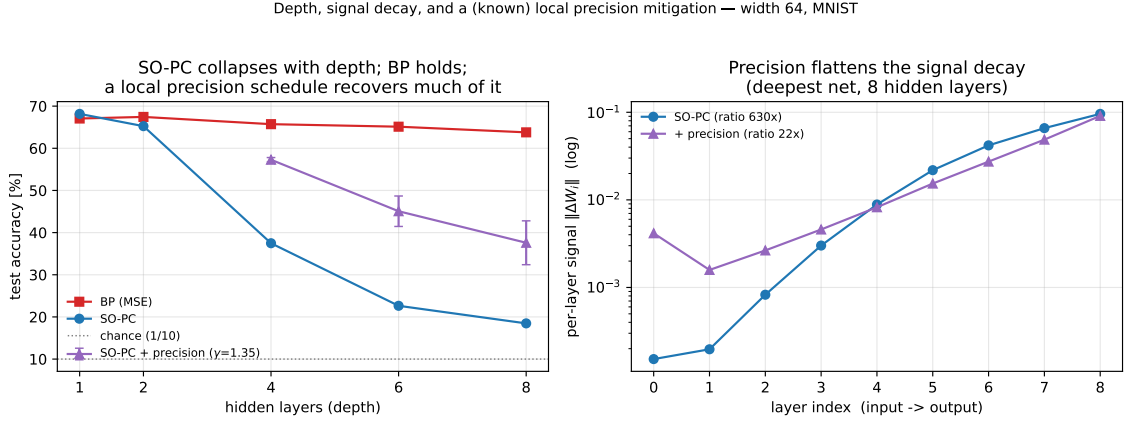


Figure 5.5: Left: test accuracy versus depth (width 64, MNIST). Standard SO-PC matches BP at one–two hidden layers but collapses as depth grows, while the matched BP baseline holds; a local precision schedule $\Pi_k = \gamma^{(n-k)}$ recovers much of the gap. Right: per-layer learning signal $\|\Delta W_i\|$ at the deepest network — it decays exponentially toward the input under standard SO-PC (output/input ratio $\sim 669\times$) and is flattened $\sim 10\text{--}30\times$ by the precision schedule.

Table 5.6: Test accuracy (%) versus depth (width 64, MNIST, 3 seeds). SO-PC collapses with depth while matched BP holds; a local precision schedule recovers much of the gap at moderate depth, with the optimal γ falling as the γ^{depth} gain compounds.

hidden layers	BP	SO-PC	SO-PC + precision ($\gamma=1.35$)
1	67.1	68.2	—
2	67.4	65.2	—
4	65.7	36.0	57.3
6	65.1	25.2	45.1
8	63.8	21.4	37.6

reproduces the established phenomenon (Goemaere et al., 2025; Qi et al., 2025) on MNIST.

5.8.2 A local precision schedule mitigates it — but is a known technique

A purely local, per-layer precision gain $\Pi_k = \gamma^{(n-k)}$ (input-side layers given higher precision, amplifying their decayed error during settling) recovers a large part of the collapse: at a single safe gain $\gamma = 1.35$, from 36/25/21% to 57/45/38% at depths 4/6/8 (Table 5.6), robustly across three seeds. It is *mechanistically* fixing the decay — it flattens the per-layer signal profile by $10\text{--}30\times$ (output/input ratio $\sim 669\times \rightarrow 22\times$ at depth eight) — rather than coincidentally helping accuracy. A higher gain

recovers even more at shallow depth (a sweep gives $\gamma = 1.5 \rightarrow 62/56\%$ at depths 4/6) but *destabilises* at depth eight, exactly the stability-versus-compensation trade-off expected from the compounding γ^{depth} gain: the optimal γ must fall as depth grows.

We stress that this is *not* a novel fix. Layer- and time-dependent precision schedules for deep PC are published (Qi et al., 2025; Innocenti et al., 2025), where they restore backpropagation-equivalent performance beyond seven layers — more effectively than the single-parameter schedule used here. The error-optimisation reparametrisation (Goemaere et al., 2025) removes the decay outright by transporting the signal with global backpropagation, at the cost of the local, parallel structure that motivates our kernel (Chapter 6). Our contribution here is the clean reproduction and the honest mapping of where a local mitigation does and does not suffice, not the mitigation itself.

Chapter summary. The fused kernel is a real but bounded engineering win ($3.2\times$ at small batch, an architectural crossover at large batch), the first of its kind for PC and now general-depth and used as the engine of a real study. The fair PC-vs-BP comparison is a clean null — $\text{PC} \approx \text{BP}$ at matched loss across every regime tested — plus the exposure of three confounds; the Song-faithful replication, at ten seeds, finds no $\geq 1\sigma$ gap at any budget. A generatively trained PC recovers the “one mechanism, many tasks” property that a discriminative PC cannot. We interpret these results, and what they mean for the field, in Chapter 6, and bound them in Chapter 7.

Chapter 6

Discussion

The preceding chapter reported what we measured. This chapter asks what those measurements *mean*. We are deliberately careful here, because the central findings of this study are largely null — under a fair comparison, our from-scratch State-Optimization predictive coding (PC) network neither beats nor is beaten by back-propagation (BP) at MNIST/FashionMNIST scale — and a null result is precisely the kind of result that invites overinterpretation in both directions. We therefore separate four distinct interpretive questions and treat each on its own terms: whether PC’s reputed advantages are real (Section 6.1), why a single methodological checklist explains the literature’s disagreement and transfers beyond PC (Section 6.2), what the systems measurements teach us about *where the compute actually goes* in PC inference (Section 6.3), what a generative PC genuinely buys that a discriminative one does not (Section 6.4), and how an adversarial-verification habit caught a framing error before it reached the page (Section 6.5). We close with an explicit accounting of threats to the validity of the inferences drawn here (Section 6.6). Throughout, the substantive enumeration of what this study *cannot* claim is deferred to Chapter 7; the present chapter is about interpretation, not caveats.

6.1 Are Predictive Coding’s Advantages Real?

Predictive coding is advertised in the literature as a local-learning alternative to backpropagation that, beyond its biological plausibility, is empirically *better* along several axes: it is said to approximate BP’s gradients while needing fewer samples, to forget less in continual-learning settings, to be more robust, and to do all of this with a single settling mechanism that also doubles as a generative model. Our fair-comparison study (Sections 5.3 and 5.4) found none of the discriminative-side advantages to survive once the comparison is made honestly. The headline verdict

is parity: at matched loss, with per-method-tuned learning rates and bootstrap confidence intervals over multiple seeds, PC and BP are statistically indistinguishable on bulk accuracy, the confidence intervals overlap, and the directional residue — slightly worse noise robustness for PC, a hair’s-breadth edge in the faithful interference replication — never exceeds one standard deviation.

6.1.1 What “parity” does and does not mean

It is worth stating the verdict with care, because “ $PC \approx BP$ ” is easy to misread. We are *not* claiming that PC and BP are the same algorithm. They are demonstrably not: PC settles the free states to a (near-)equilibrium before touching the weights, whereas BP computes a single backward pass and updates immediately, and the two procedures produce genuinely different intermediate activity (this is the substance of Song et al.’s “prospective configuration” argument, Song et al., 2024). What we are claiming is narrower and more defensible: along the *outcome* metrics we measured — test accuracy, noise robustness, sample efficiency, and forgetting in continual learning — the distinction between the two learning rules does not manifest as a measurable performance gap once confounds are removed. The two rules arrive at comparable solutions by different routes. That a different mechanism reaches the same place is itself informative; it does not license the stronger claim that the mechanisms are equivalent, only the claim that, at this scale and on these tasks, the mechanism does not buy a performance advantage.

Equally, parity is a *scoped* claim, not a universal one. Our measurements live at MNIST/FashionMNIST scale with shallow (two-to-five-layer) networks. The honest statement is that at this scale, against a fairly tuned baseline, no PC advantage exceeds noise — not that no advantage can exist anywhere. The regimes in which PC is most strongly theorized to differ — deep networks where the prospective-configuration argument compounds across layers, genuine online single-example streams, reinforcement learning — are exactly the regimes our setup does not stress. This is the correct frontier to defer to (Chapter 7), and we are careful not to overclaim past it.

6.1.2 Why the literature disagrees

If PC’s advantages are not real at this scale, why does so much of the literature report them? Our answer, and one of the transferable contributions of this study, is that the inconsistency is not mysterious: naive comparisons trip over a small number of recurring confounds, and each confound manufactures a spurious PC advantage

in a predictable direction. We identified three (developed in Section 6.2), and in our own pre-confound runs all three fired. Our first PC-versus-BP table showed an apparent BP *win* on bulk accuracy and an apparent PC *win* on continual learning; the adversarial methodology review showed both to be artifacts. The bulk gap was almost entirely the loss function — cross-entropy versus mean-squared error to a one-hot target — not the learning rule, since the loss effect alone (BP-CE minus BP-MSE) accounts for the bulk of a naive “BP beats PC” gap. The continual-learning gap was a double artifact: comparing against a cross-entropy baseline (which forgets more because softmax produces large, aggressive output updates) *and* reading the forgetting metric without controlling for the fact that PC learns each task less well in the first place, so it has mechanically less to forget.

This reframing reconciles what looks like a contradiction in the background literature. On the one hand, a line of theoretical work shows that PC *approximates* BP under restrictive conditions — the fixed-prediction regime of Whittington and Bogacz (2017), the arbitrary-graph result of Millidge et al. (2022), and the limiting analyses of Innocenti et al. (2026). On the other hand, Song et al. (2024) argue that *without* those constraints, energy-based networks follow a distinct prospective-configuration dynamics that yields advantages. Both can be true: PC collapses toward BP in the small-step, near-feedforward-initialization limit, and departs from it when the settling is allowed to run to genuine equilibrium. Our finding sits on top of this: even in the unconstrained, fully-settled regime where PC *should* differ most, the difference does not translate into a measurable advantage over a *fairly tuned, loss-matched* BP at our scale. The literature’s disagreement is, on this reading, less about PC’s true behaviour and more about how carefully the BP baseline was constructed.

6.2 The Confound Checklist as a Transferable Contribution

The most reusable product of this study is not the parity verdict itself but the diagnostic machinery that produced it. We distilled the failure modes of naive PC-versus-BP comparisons into a three-item checklist. We argue that this checklist is the genuine contribution of the methodological strand of the thesis, because it generalizes well beyond PC: it applies to *any* comparison between a biologically-motivated learning rule and backpropagation, and indeed to many “alternative-versus-baseline” empirical claims in machine learning.

6.2.1 The three confounds

Confound 1: the loss function. The single largest hidden variable is the objective being minimized. A PC network that clamps a one-hot target onto its linear output state and minimizes squared prediction error is minimizing mean-squared error (MSE); a BP baseline that minimizes `CrossEntropyLoss` on logits is optimizing a different loss surface entirely. These are not interchangeable: softmax cross-entropy is well known to outperform MSE-to-one-hot on classification. Comparing PC-MSE against BP-CE therefore conflates the learning rule with the loss, and because the loss effect is large (Section 5.3), most of a naive gap is the objective, not the mechanism. The fix is to add a matched BP-MSE arm so that the valid like-for-like comparison is PC-MSE versus BP-MSE, while the difference BP-CE minus BP-MSE quantifies, and quarantines, the pure loss effect.

Confound 2: the stability–plasticity trade-off. In continual learning, “forgets less” is not the same as “retains better.” Backward transfer measures the drop in accuracy relative to the freshly-learned diagonal; a model that writes weaker, more diffuse task representations has a lower diagonal and therefore mechanically less to lose, yielding a less-negative backward-transfer score that flatters it. A forgetting claim is only defensible when read jointly against two controls: the per-task learn-accuracy (the diagonal, i.e. how well each task was acquired) and the retained absolute accuracy on the first task after all tasks (the sharpest single control, because it sidesteps the diagonal entirely). If PC “forgets less” only because it learns less, the apparent advantage is a relabelling of underfitting.

Confound 3: the untuned baseline. A comparison is only as honest as its weakest baseline. We found that an untuned BP-MSE baseline could sit stuck at chance on a class-incremental split, manufacturing a false PC “win” that a learn-accuracy gate immediately exposed. The discipline this enforces is twofold: the baseline’s learning rate must be tuned on the same objective and validation protocol as the method under test, and PC’s own state and weight learning rates (η_x and η_w) must be tuned *jointly*, because they are coupled by a stability frontier — sweeping η_w alone, at a fixed and too-large η_x , drives PC into divergence and systematically understates it.

6.2.2 Why the checklist transfers

None of these three confounds is specific to predictive coding. The loss confound bites whenever two methods are compared under different objectives; the plasticity

confound bites whenever a retention metric is read without a learning control; the baseline confound bites whenever the comparison point is left untuned. The checklist is thus a lightweight, reusable instrument for reviewers and authors in an empirically confounded subfield: before believing a “method X beats backpropagation” claim, ask whether the loss was matched, whether the retention metric was controlled for plasticity, and whether the baseline was actually tuned to learn. We offer it in that spirit — as a portable artifact of the methodology, more durable than any single number in Chapter 5.

6.3 The Systems Insight: Launch Overhead and the GEMM Trade-off

The systems strand of the thesis produced the project’s only unambiguous “better” result — an engineering win — and it carries an interpretation worth drawing out independently of the speedup figures. The measurement, not the speedup, is the contribution.

6.3.1 PC inference is launch-overhead-bound, and that is a framework tax

The headline systems observation (Section 5.2) is that the PyTorch State-Optimization backend spends roughly half a millisecond per settling step *independent of batch size*, with the GPU only partially utilized. The mechanistic evidence is direct: a launch-count profile shows the PyTorch backend issuing thirty-one kernel launches per settling step (scaling linearly with the number of settling steps T), whereas the fused kernel issues exactly one regardless of T . The correct interpretation is that, at MNIST scale, PC inference is *not* compute-bound; it is bound by kernel-launch overhead — the cost of dispatching many tiny operations from the host. This is a recognized “framework tax” on workloads built from many small operations, and we are careful to claim neither the observation nor the remedy as novel in general. The fused, persistent, states-resident single-launch technique is established prior art, originating in cuDNN’s persistent RNN and Baidu’s `persistent-rnn` (Diamos et al., 2016) and reappearing in recent LLM megakernels. What *is* new, to our knowledge, is the transfer of that technique to the PC settling loop and the measurement that pins the bottleneck: an audit of eleven open PC libraries and the entire Equilibrium-Propagation family found none shipping a hand-written CUDA settling kernel (Chapter 3; the audit is the

substance of the novelty claim). The defensible statement is therefore narrow — “first hand-written fused CUDA settling kernel *for PC*, to our knowledge” — and we hold it at that width deliberately.

6.3.2 The architectural trade-off bounds the win

The more interesting systems insight is *why* the kernel wins only in a bounded regime. At small and medium batch, fusing the T -step loop into one launch eliminates the launch overhead and delivers a real speedup. But at large batch the bottleneck shifts from launch overhead to raw compute, and here the design that wins at small batch actively works against you. A fused-resident kernel holds the layer states on-chip across all T steps; that residency consumes precisely the register and shared-memory budget that a fast, register-blocked general matrix-multiply (GEMM) needs to approach the hardware’s floating-point peak. The fused-resident design and a cuBLAS-class GEMM are thus *architecturally opposed*: you cannot have both maximal residency and maximal GEMM throughput in the same kernel, so at large batch the vendor library’s register-blocked GEMM wins and our naive in-kernel GEMM cannot keep up. This is not merely “not yet optimized”; it is a structural trade-off, and recognizing it as such is itself a reportable finding. Batch-tiling and caching the clamped input activation push the crossover outward, but they do not abolish the trade-off.

6.3.3 Why this matters for PC specifically

The trade-off is not a defect for PC’s purposes, because the regime where the kernel wins — small-to-medium batch — is exactly the regime PC most naturally inhabits. PC is biologically motivated as an online or near-online learner, and the prospective-configuration advantages are theorized to appear in small-batch and single-example streams. The large-batch compute regime, where cuBLAS is correctly the right tool, is the regime *least* characteristic of PC. The practical upshot is that the win lands where it is useful: when we ran the kernel as the engine of our own faithful Song-exact study at batch 32 (Section 5.4), it delivered a meaningful end-to-end acceleration at per-seed-identical results, and its generalization to arbitrary depth (validated to roughly 10^{-6} across depths two through five) means it is not a single-topology curiosity. This is what ties the systems contribution to the science: the kernel is the measured, validated engine of a real experiment, not a microbenchmark presented in isolation.

6.4 What a Generative PC Buys

One of PC’s most cited selling points is the “one mechanism, many tasks” property: the same settling rule that classifies should also generate, inpaint, and detect anomalies, with only the clamping pattern changing. Our results (Section 5.5) make a sharp distinction here that we believe is the correct interpretation, and it is easy to get wrong.

6.4.1 The orientation, not the mechanism, is decisive

A *discriminatively* trained PC — the standard supervised setup that clamps input and label and settles the hidden states to learn an input-to-label map — cannot generate. Clamping a label and settling the input yields noise, not a recognizable digit, and residual energy fails to separate in-distribution from out-of-distribution inputs. This is architectural, not a tuning failure: a discriminatively trained network has learned a map from images to labels, not a generative model of images, so settling “backwards” lands on no learned image manifold. The remedy is to flip the orientation and train *generatively* (label to image, clamping label and image while the hidden states settle). With that single change, the identical settling rule that classified now produces recognizable per-class prototypes, reconstructs occluded halves, and separates uniform-noise out-of-distribution inputs with an anomaly AUC of 1.0. The lesson is that the celebrated “one mechanism, many tasks” property is real, but it is a property of a *generatively trained* PC, not of the discriminative network most classification benchmarks use. The mechanism is the same; the training objective is what unlocks the multi-task capability.

6.4.2 Honest reading of the capability

We are careful not to oversell what the generative PC delivers. With a one-hot label input there are at most ten distinct outputs, so the “generation” is a set of class *prototypes* — the per-class mean image — not a diverse sample from a learned distribution; genuine sampling diversity would require a continuous latent and a prior, as in a variational autoencoder. Inpainting is prototype-driven and therefore blurry rather than pixel-sharp. And the anomaly result, while clean, is measured against uniform noise, which is an easy out-of-distribution test; a near-OOD probe such as FashionMNIST against MNIST, or rotated digits, would be a substantially harder and more informative test. The defensible interpretation is therefore: a generative PC genuinely exhibits the multi-task property from one settling rule, and the perfect anomaly separation is a real energy-based signature (the model assigns

high residual energy to inputs it cannot generate); but the qualitative outputs are prototypes, and the OOD test is gentle. This is a demonstration of the property, scoped honestly, not a competitive generative model.

6.5 Methodology: Adversarial Verification

A recurring theme of this study is that the most dangerous errors in an empirically confounded subfield are not coding bugs — those are caught by tests and gradient checks — but framing errors: subtle misreadings of what a prior result claims, or what one’s own measurement isolates. These are dangerous precisely because they survive a working pipeline. We adopted a lightweight safeguard against them, and we argue it is reusable enough to be a methodological contribution in its own right.

6.5.1 The three-lens review and the error it caught

Before committing a finding to the writeup, we subjected it to an adversarial review along three independent lenses: a *literature* lens (does this faithfully represent what the cited work claims?), a *statistics* lens (do the intervals, seed counts, and tests support the strength of the claim?), and an *alternative-explanations* lens (what confound could produce this result without the mechanism we are attributing it to?). Applied to our Song-exact alternating replication (Section 5.4), this review caught a genuine framing error before it reached the page. Our first draft attributed a learning-rate-robustness claim to Song’s interference figure; the literature lens established that the figure in question is a *less-interference* claim measured with the learning rate optimized *per method*, while the learning-rate-robustness claim lives in a *different* figure. The correction was substantive: it changed what we tuned (we tune the learning rate per method, as the interference claim requires) and what we report (we add the worse-of-the-two-tasks accuracy as the interference tell, alongside the mean). Had the error survived, we would have run and reported the wrong experiment while believing we were replicating Song.

6.5.2 Why this is a reusable safeguard, not a one-off

The same three-lens discipline repeatedly corrected our own over-eager conclusions: the statistics lens caught a confidence-interval mislabelling (68% intervals reported as 95%), and the alternative-explanations lens is exactly what surfaced the three confounds of Section 6.2. We present adversarial verification not as a heroic one-time audit but as a cheap, repeatable habit: for any empirical claim, ask whether it

misreads the literature, whether the statistics carry it, and whether a confound explains it just as well. In a subfield where “method X beats backprop” claims are easy to make and hard to reproduce, this is a low-cost, high-yield form of self-skepticism. It is, in a sense, the methodological complement to the confound checklist: the checklist enumerates the known traps, and the three-lens review is the procedure for finding the ones not yet on the list.

6.6 Threats to Validity

Interpreting a null result responsibly requires being explicit about what could undermine the inferences above. We organize the threats along the standard internal/external/construct axes and state, for each, how it was mitigated and what residual risk remains. The substantive enumeration of the study’s scope limits — scale, depth, prototype-only generation, the bounded kernel regime — belongs to Chapter 7; here we restrict ourselves to threats to the *validity of the conclusions* drawn in this chapter.

6.6.1 Internal validity

Internal validity concerns whether the measured differences (or non-differences) are caused by the variable we intend to manipulate — the learning rule — rather than by an uncontrolled nuisance. This is the axis the three confounds attack directly, and our principal mitigation is the fair-comparison protocol itself: the BP baseline clones the PC network’s initialization and uses the identical forward function, so only the learning rule differs; the loss is matched via the BP-MSE arm; learning rates are tuned per method on a held-out validation split and reported on test; and PC’s η_x and η_w are tuned jointly to respect the stability frontier. The residual internal threats are honestly acknowledged: our headline used a modest seed budget with bootstrap intervals, and although we raised the faithful replication to ten seeds — which hardened rather than overturned the verdict — a non-significant directional effect at this seed count cannot be ruled to be exactly zero, only ruled to be within noise. The learning-rate grids are finite, so we cannot exclude that a finer grid would shift a point estimate, though the joint-tuning frontier we mapped makes a large unmodelled improvement unlikely. We also note that the autonomous hyperparameter search (Section 5.7) independently rediscovered the $\eta_x \leftrightarrow \eta_w$ interaction, which serves as an internal consistency check on the tuning.

6.6.2 External validity

External validity concerns how far the conclusions generalize. Here the threats are real and we do not minimize them. All measurements are at MNIST/FashionMNIST scale with two-to-five-layer networks; the parity verdict is scoped to that regime, and we explicitly decline to extend it to deep networks, genuine online streams, or larger datasets, which is where PC is most theorized to differ. The faithful Song replication, even at its most faithful, still differs from the original in scale and seed budget, so our null does not globally refute Song — the honest claim is the scoped one. The single-GPU benchmarking (one RTX 3080 Ti) means the systems crossover points are hardware-specific; the *shape* of the launch-overhead-versus-compute trade-off is general, but the exact batch sizes at which the kernel wins or loses would move on other hardware. These threats are mitigated by scoping every claim to its measured regime rather than eliminated, and they motivate the future-work directions of Chapter 7.

6.6.3 Construct validity

Construct validity concerns whether the metrics we chose actually measure the constructs we care about. The risk is most acute in continual learning, where “forgetting” is a construct that a single backward-transfer number measures poorly — which is the entire point of the plasticity confound. Our mitigation is to report forgetting as a triple (backward transfer, learn-accuracy, and retained first-task accuracy) so that the construct is triangulated rather than reduced to one potentially misleading scalar, and we added an EWC positive control that forgets less by construction, confirming the harness measures forgetting correctly. For the anomaly-detection capability, AUC against uniform noise is a valid but weak operationalization of “out-of-distribution detection,” and we flag the easy-OOD caveat rather than letting the AUC of 1.0 stand unqualified. A subtler construct issue is that PC and BP differ in their *inference* procedure — PC settles the output at test time, BP reads a single forward pass — so noise-robustness and continual comparisons partly measure inference dynamics, not only plasticity; we treat this as a documented feature of PC rather than a flaw, but flag it so the reader knows the forward path is not perfectly matched across arms. The recurring mitigation across all three axes is the same one that defines the study: state every claim with its interval, its seed count, and its regime, and let the sober framing do the work.

Chapter 7

Limitations and Future Work

A study whose headline contribution is methodological honesty owes the reader an equally honest account of its own boundaries. The findings reported in Chapter 5 are, for the most part, deliberately sober: at the scale we study, a fairly tuned predictive-coding (PC) network is statistically indistinguishable from a matched backpropagation (BP) baseline, and our systems contribution—a fused CUDA settling kernel—wins only in a delimited batch regime. None of these results should be over-read. This chapter therefore does two things. First, it states precisely what our claims do *not* entail: the scale at which they hold, the seed budgets behind the confidence intervals, the architectural regimes in which the kernel helps or hurts, and the exact provenance of the “first-for-PC” novelty claim. Second, it lays out the concrete extensions that would either widen the scope of the present claims or test the hypotheses that this study could only flank rather than resolve. The intent throughout is to leave a reviewer with no surprises: every caveat that an adversarial reader might raise is raised here first, by us.

7.1 Limitations

We group the limitations into four families: those of *scientific scope* (what the null result does and does not establish), those of *the systems contribution* (where the kernel wins and where it does not, and how far the novelty claim reaches), those of *the capability demonstrations* (the generative model, the anomaly task, the incremental-PC variant), and those of *experimental reach* (scale and architecture). We treat each in turn.

7.1.1 Scope of the scientific findings

The null result is scale- and budget-bounded, not universal

The central empirical claim of this thesis is a *null*: across bulk accuracy, noise robustness, sample efficiency, domain-incremental learning, and class-incremental learning, a loss-matched and per-method-tuned PC network shows no advantage over BP that exceeds our measured noise (Section 5.3). It is essential to state what this claim is *not*. It is not a proof that PC can never differ from BP. It is the considerably weaker—and we believe defensible—statement that *at MNIST and FashionMNIST scale, with a fairly tuned baseline and a matched loss function, no PC advantage exceeds the seed-to-seed noise*. The distinction matters because the literature reports advantages in regimes we did not reach, and the theory (Innocenti et al., 2026) locates PC’s distinctive behaviour precisely in regimes (depth, width) that our [784, 256, 256, 10] network does not probe and that the theory itself predicts to become unstable as they scale. Our result is a clean data point at one scale, not a refutation of the field.

The null does not globally refute Song et al.

This caveat deserves its own treatment because it is the most likely point of misreading. Our class-incremental experiments (Section 5.3) follow the *regime* studied by Song et al. (2024), and Section 5.4 reports an architecture-faithful replication of their *exact* alternating Split-FashionMNIST protocol: a four-layer 32–32 sigmoid network with Xavier-normal initialisation, two disjoint five-class tasks sharing one five-output head, trained by alternating at the minibatch level. On the axes Song specifies (depth, hidden width, activation, initialisation, task split, shared head, alternation) we match the published method. Even so, two honest gaps remain. First, the *scale and seed budget* differ: we ran $n = 10$ seeds and reported 68% (1σ) bootstrap intervals, whereas the original study operates at a different scale with its own seed budget; a 1σ null at one scale is not a 95% null at another. Second, the original 84-iteration training budget leaves every method at chance in our hands—a faithfulness-versus-trainability tension that we resolved by treating the training budget as a controlled axis from 84 iterations to convergence, rather than by silently enlarging it. Within that swept budget we find no PC-versus-BP(MSE) difference exceeding 1σ at *any* point, and raising the seed count from five to ten did not promote the directional trend (PC slower early, marginally ahead at convergence) to significance. The honest reading is therefore: *our most faithful replication agrees with our parity verdict rather than overturning it*, while the residual differences in

scale and budget mean we cannot claim to have reproduced or refuted the original finding in its own terms.

A corrected framing, not an original interpretation

The interpretation we report in Section 5.4 is itself the product of an adversarial three-lens review (literature faithfulness, statistics, alternative-explanations) that caught an earlier draft misattributing a learning-rate-robustness claim to Song et al. (2024)’s interference figure. The original figure is a *less-interference* claim with the learning rate optimised per method, not an LR-robustness claim. We mention this here because it is a limitation of method as much as a strength: it demonstrates that, absent the adversarial check, we would have compared against the wrong baseline metric. The safeguard is reusable and we recommend it, but its necessity is also a reminder that single-pass readings of the literature in this field are error-prone, our own included.

The confound checklist explains, but does not exhaust

We identify three confounds that manufacture spurious PC advantages in naive comparisons (Section 5.3): the loss function (cross-entropy versus MSE, worth +5.9 percentage points of bulk accuracy on its own), the stability–plasticity trade-off in forgetting metrics, and an untuned baseline learning rate. This list is offered as an explanation for the literature’s inconsistency, not as a closed taxonomy. Other confounds—initialisation schemes, data ordering, the choice of optimiser for the baseline, the number of settling steps T relative to network depth—could equally distort a comparison, and we controlled rather than exhaustively ablated several of them. The checklist is a practical aid, not a completeness theorem.

7.1.2 Limitations of the systems contribution

The kernel wins only in the small/medium-batch regime

The fused settling kernel (Section 4.3, Section 5.2) is honestly a *regime-specific* accelerator, not a general-purpose replacement for the PyTorch backend. PC inference at MNIST scale is launch-overhead-bound: the PyTorch backend issues 31 kernel launches per settling step and runs at roughly 0.5 ms/step independent of batch size, leaving the GPU 25–49% utilised. Fusing the entire T -step loop into a single launch removes that overhead and yields $3.2\times$ at batch 64. But the same property that makes the fused design fast at small batch—holding the layer states resident on-chip across all T steps—is architecturally opposed to the register-blocked tiling

that a cuBLAS-class GEMM needs: residency consumes precisely the register and shared-memory budget a fast matrix multiply requires. Consequently, batch-tiling and input-activation caching push the crossover only to roughly 1024–1500, and at batch ≥ 2048 , where *compute* rather than launch overhead dominates, cuBLAS wins and the correct backend is `pytorch`. We report this not as “not yet optimised” but as a genuine systems finding—the fused-resident and tiled-GEMM designs are in tension—yet it is unambiguously a limitation on the kernel’s applicability.

The large-batch tiled path is still depth-two

The kernel now exists in two forms with different generality. The per-sample path (one thread block per sample) has been generalised to *arbitrary depth*: the per-layer weights, states, and biases are passed as device-pointer arrays with shared-memory offsets computed at launch, and the result is validated against the PyTorch equilibrium to roughly 10^{-6} across depths two through five (one to four hidden layers), for tanh and sigmoid activations and batches of 32 and 256. The batch-tiled path (the one that buys large-batch weight reuse), however, remains specialised to the two-hidden-layer topology. The general-depth kernel is therefore restricted to the small-batch, launch-bound regime—which is the regime in which it wins, so the restriction is not crippling—but a register-blocked tiled GEMM for the large-batch *and* deep regime is not implemented. Lifting the depth specialisation on the tiled path is reserved future work.

“First for PC” carries the to-our-knowledge caveat

The novelty claim is stated carefully and we restate its exact scope here. To our knowledge, this is the first hand-written, fused, single-launch CUDA kernel that runs the entire T -step PC settling loop with layer states held resident on-chip across steps. This is supported by a direct source audit of eleven open PC libraries (PCX, JPC, ngc-learn, μ PC, PRECO, Torch2PC, pypc, pybrid, ePC, and Song’s own repositories), none of which ship a custom CUDA settling kernel—they are uniformly JAX/JIT or PyTorch-autograd—and by an additional audit of the Equilibrium-Propagation family, the closest algorithmic relative, which likewise ships no hand-written CUDA settling kernel (the one EP repository with hand-written GPU kernels uses per-step elementwise Triton kernels with the settling iteration on the host). Two qualifications are mandatory. First, the *technique*—a fused, single-launch, states-resident persistent kernel—is established prior art, dating to cuDNN’s persistent RNN and Baidu’s `persistent-rnn` (Diamos et al., 2016) and re-applied in recent LLM megakernels; our contribution is its transfer to, and measurement on, the

PC settling loop, not the invention of the pattern. The closest fixed-point analogue, deep equilibrium models (Bai et al., 2019), and the closest PC-side work, error-optimisation PC (Goemaere et al., 2025), attack related goals without a custom kernel and are cited and delimited accordingly. Second, a negative existence claim over unindexed or industrial code cannot be proved; the “to our knowledge” caveat is therefore not a rhetorical hedge but a literal statement of the evidence’s limit. The observation that PC inference is launch-overhead-bound is itself a generic “framework tax” and is *not* claimed as a contribution.

7.1.3 Limitations of the capability demonstrations

Generative output is class prototypes, not diverse samples

The generative PC of Section 5.5 genuinely recovers the “one mechanism, many tasks” property: the same settling rule classifies, generates, inpaints, and detects anomalies, depending only on what is clamped. But the generative model maps a one-hot label to an image, so its ten possible inputs produce at most ten distinct outputs: recognisable *per-class prototypes* (effectively the per-class mean), not a diverse sample distribution. Inpainting is correspondingly prototype-driven and blurry rather than pixel-sharp. Producing diverse samples would require a continuous latent with a prior (a VAE-like construction), which we did not build. The demonstration establishes the qualitative capability, not generative fidelity.

Uniform noise is an easy out-of-distribution test

The anomaly-detection result—an AUC of 1.0 separating in-distribution MNIST (residual energy 5.1) from uniform-noise inputs (energy 94.5)—is real but must not be over-read. Uniform noise is an *easy* out-of-distribution (OOD) signal: it lies far from any plausible image manifold, so perfect separation is a weak test of the detector. A genuinely informative evaluation would use *near*-OOD inputs, such as FashionMNIST presented to an MNIST-trained model, or rotated digits, where the OOD distribution shares low-level statistics with the training data. We report the easy result honestly and flag the harder test as the meaningful one.

Incremental PC is learning-rate sensitive

The incremental-PC (iPC) variant (Salvatori et al., 2024) of Section 5.3 interleaves a weight update with every settling step rather than updating once at equilibrium. Because it applies T weight updates per minibatch, its effective weight learning rate is roughly T times larger, and at the standard rate it diverges. At a rate scaled down

by approximately $1/T$ it is stable and, in a fast-training regime, outperforms standard PC (74.3% versus 63.1% on a short MNIST run). The capability is therefore real but conditional: iPC trades robustness to the learning rate for faster per-step learning, and a practitioner who omits the $\sim \eta_w/T$ rescaling will simply see it diverge. We report it as an optional extension with this sensitivity made explicit, not as a drop-in improvement.

Precision schedules remain reserved

Throughout this work the precision (inverse-variance) weighting Π of the prediction errors is fixed to the identity, which reduces the energy $E = \frac{1}{2} \sum_k \|\varepsilon_k\|^2$ to a plain sum of squared errors—the modern default of PCX and JPC (Pinchetti et al., 2024; Millidge et al., 2024). Learnable or scheduled precisions are a documented dimension of the PC objective and a plausible lever on both the depth-scaling problem (precision-weighted gradients are one of the proposed fixes) and the energy imbalance across layers, but we did not implement them. The “precision schedule” search dimension is therefore reserved, and any claim we make is implicitly conditioned on $\Pi = I$.

7.1.4 Limitations of experimental reach

Scale and the gap to Transformers

The entire study lives at MNIST and FashionMNIST scale with a vanilla state-optimisation PC network. We do not demonstrate PC on larger datasets, on convolutional or attention architectures, or at anything approaching the scale at which the field’s open questions (Chapter 3) actually bite. The *scale gap*—PC has not been shown to work at Transformer scale—is untouched here and is frontier-laboratory work; our contribution is a clean, well-characterised building block, not a scaling result.

Depth scaling is sidestepped, not solved

Closely related, the depth-scaling failure of PC—the observation that deeper PC-trained networks are often *worse*, the opposite of BP, driven by an exponential signal-decay problem (Goemaere et al., 2025)—is *characterised but not solved* here. We reproduce it cleanly (Section 5.8): standard SO-PC matches BP at one–two hidden layers but collapses to near-chance by eight, with the per-layer learning signal decaying $\sim 669\times$ from output to input. We also show that a strictly local precision schedule $\Pi_k = \gamma^{(n-k)}$ mechanistically mitigates it (flattening the signal

profile 10–30 $\times$ and recovering much of the accuracy at moderate depth), but only partially at depth eight, and—crucially—this is a *known* technique (Qi et al., 2025; Innocenti et al., 2025), not our contribution, and it is bettered by those works and by error optimisation (Goemaere et al., 2025). What remains genuinely open for us is full deep-PC *trainability*: matching BP at the depths where the published fixes operate, which is frontier-laboratory work. The problem is now mapped at our scale, not resolved.

Single optimiser regime and offline evaluation

Finally, the comparisons use a single optimisation regime (plain local updates for PC, SGD-style updates for the matched BP baseline) and an offline, batch-trained evaluation. We do not study the strict online (batch-size-one) regime that is among PC’s most cited putative niches, nor optimiser interactions (momentum, adaptive methods) that could shift the parity verdict in either direction. All experiments are implemented in pure PyTorch with hand-derived updates verified against autograd as an external oracle, and the full suite comprises forty offline tests; this gives correctness confidence but is not a substitute for the broader sweep just described.

7.2 Future Work

The limitations above map naturally onto a future-work programme. We organise it by the same two threads as the thesis—the systems contribution and the scientific contribution—and add the capability extensions that the generative results invite. Where a direction is squarely frontier-laboratory work we say so; where a solo, small-scale project can make a real dent we say that too.

7.2.1 Extending the systems contribution

A register-blocked tiled GEMM and an ahead-of-time build

The clearest engineering next step is to attack the large-batch regime directly. The kernel currently loses to cuBLAS at batch ≥ 2048 because its in-kernel matrix multiply uses scalar fused-multiply-adds with no register blocking and reaches only a fraction of the FLOP peak. A register-blocked, shared-memory-tiled GEMM—in effect reimplementing the core of cuBLAS inside the fused kernel—would, if combined with a careful budgeting of the residency-versus-tiling trade-off described above, narrow or close the large-batch gap. This is a substantial undertaking and arguably of questionable return given that cuBLAS already serves that regime well, but it

is the principled way to lift the fused design beyond its current ceiling. In parallel, replacing the current just-in-time `torch.utils.cpp_extension.load` build with an ahead-of-time (AOT) `setuptools/CUDAExtension` build against the ABI-stable LibTorch API would improve reproducibility and packaging. Lifting the batch-tiled path to arbitrary depth (it remains depth-two) is the companion task.

An Nsight Compute profile to corroborate the launch-overhead account

Our mechanistic evidence that launch overhead is the addressed bottleneck rests on a profiler launch count (the PyTorch backend issues 620, 1240, or 2480 launches for $T = 20, 40, 80$ versus the fused kernel’s single launch). A full Nsight Compute report—occupancy, achieved bandwidth, memory-versus-compute roofline placement, warp-stall reasons—would harden this account against the “solver-view” objection raised by ODE-solver-based libraries such as JPC and ePC, demonstrating at the hardware-counter level that the PyTorch path is launch-bound and the fused path is not. We note the launch-count figure is already presented in Section 5.2; the Nsight report is the natural deepening.

7.2.2 Extending the scientific contribution

An Equilibrium-Propagation comparison arm

Equilibrium Propagation (EP) (Scellier and Bengio, 2017; Laborieux et al., 2021) is the closest algorithmic relative of PC: both relax to an equilibrium and derive a local weight update from it. We audited EP for the novelty claim but did not implement it as a comparison arm. Adding an EP arm under the same fair-comparison protocol—matched architecture, initialisation, data, and loss; per-method tuning; bootstrap confidence intervals—would situate our PC-versus-BP parity verdict within the broader energy-based-learning family and test whether EP, too, collapses to BP-like behaviour at this scale or carves out a distinct regime.

Probing where PC and BP diverge

The deepest open question we can only flank is the $PC \approx BP$ equivalence puzzle: the theory (Innocenti et al., 2026) holds that the width- and depth-stable feature-learning parametrisations of PC coincide with those of BP, and that PC’s distinctive regime becomes unstable as it scales—leaving open what PC is ultimately *for*. A small project cannot resolve this, but it can contribute data points. Concretely, one could test whether the PC and BP *solutions* themselves diverge—do they reach different minima, exhibit different loss-landscape curvature (Zahid et al., 2023), or differ

in robustness—even when their test accuracies match. A measurable divergence in solution geometry, despite accuracy parity, would be a genuine contribution to the question, and our matched-init harness (the BP baseline clones the PC initialisation) is well suited to it.

Harder near-OOD and continual-learning baselines

On the empirical side, the anomaly task should move from easy uniform-noise OOD to near-OOD (FashionMNIST against an MNIST-trained model, rotated digits), which would turn the AUC 1.0 result into an informative measurement. On the continual-learning side, our EWC baseline (Kirkpatrick et al., 2017) already behaves as a positive control—it forgets less than the matched BP, with backward transfer of -41.1% versus -45.7% —and the natural next step is to add a replay-based baseline such as gradient episodic memory (Lopez-Paz and Ranzato, 2017) and a joint-training upper bound, ideally standardised through a continual-learning framework such as Avalanche (Lomonaco et al., 2021), to locate the absolute forgetting numbers rather than only comparing PC against BP.

7.2.3 Extending the capability demonstrations

A diverse generative latent

To move the generative PC from class prototypes to diverse samples, the one-hot label input must be replaced by a continuous latent with a prior, settled as a free variable under a generative training objective in the spirit of the original predictive-coding model (Rao and Ballard, 1999). This is a distinct model variant rather than a tuning change, and it is the most direct route to a generative PC that produces sample *diversity* and supports the sharper inpainting that prototype-driven reconstruction cannot.

Depth-scaling diagnostics and proposed fixes at small scale

Finally, on the depth-scaling problem we have already taken the small-project step of *characterising* it precisely (Section 5.8): the “deeper-is-worse” effect is reproduced across two, four, six, and eight hidden layers, the per-layer learning-signal decay is measured and plotted, and one local fix (a precision schedule) is shown to mechanistically flatten the imbalance — though it is a known technique and only partial at depth. The natural continuation is to implement error optimisation (Goemaere et al., 2025) as a fourth arm and complete the benchmark matrix (PyTorch-SO / fused-SO / EO / BP, over accuracy, wall-clock, and per-layer signal versus depth),

which would turn the present ablation into a full comparative study of *how* each method keeps deep PC trainable; reaching backpropagation-equivalent depth in a local formulation remains the frontier piece beyond it.

Taken together, these directions do not change the thesis’s sober verdict; they sharpen its edges. The systems extensions would widen the kernel’s regime of usefulness without retracting the architectural trade-off we identified; the scientific extensions would test the parity verdict against a wider family of methods and probe the one question—where, if anywhere, PC genuinely diverges from BP—that this study could only frame, not answer. That this list is long is itself in keeping with the project’s stance: an honest baseline is valuable precisely because it makes the next questions clear.

Chapter 8

Conclusion

This thesis set out to study Predictive Coding from scratch and to answer three questions about it honestly. The answers, taken together, form a sober but useful picture.

To **RQ1**—whether PC outperforms backpropagation under a fair comparison—the answer is *no, not at this scale*. Across bulk accuracy, noise robustness, sample efficiency, domain-incremental and class-incremental learning, a loss-matched and per-method-tuned PC network is statistically indistinguishable from a matched backpropagation baseline (Section 5.3). The most faithful test we could construct—an architecture-exact replication of Song et al. (2024)’s alternating Split-FashionMNIST protocol, with the training budget treated as a controlled axis and ten seeds—agrees with this parity verdict rather than overturning it: no PC-versus-BP difference exceeds one standard deviation at any budget (Section 5.4). The contribution here is not the null itself but its explanation. We isolate three confounds—the loss function, the stability–plasticity trade-off in forgetting metrics, and an untuned baseline learning rate—each of which manufactures a spurious “PC advantage” in a naive comparison, and we offer the resulting checklist as a transferable guard for the field. That the correct interpretation of the replication itself only emerged after an adversarial three-lens review, which caught us misattributing a learning-rate-robustness claim to an interference figure, is reported as both a methodological tool and a cautionary tale (Chapter 6).

To **RQ2**—where PC’s inference cost goes and whether it can be reduced—the answer is concrete. PC inference at MNIST scale is launch-overhead-bound: the framework issues tens of tiny kernel launches per settling step and leaves the GPU half-idle (Section 5.2). A fused CUDA kernel that runs the entire settling loop in a single launch, with the layer states resident on chip, removes that overhead and is—to our knowledge—the first hand-written settling kernel for PC. It delivers a

3.2× speedup at small batch, generalises to arbitrary depth, and was fast enough to serve as the actual backend of our exact-replication study, where it ran 1.45× faster end-to-end with per-seed-identical results. The same residency that makes it fast at small batch is architecturally opposed to the register-blocked tiling a cuBLAS-class matrix multiply needs, so beyond batch ~ 2048 the standard backend wins—a genuine systems finding, not a missing optimisation.

To **RQ3**—whether one PC mechanism serves many tasks—the answer is a qualified yes, with an important precondition. A *discriminatively* trained PC cannot generate; only when the network is oriented generatively (label→image) does the single settling rule yield classification, class-conditional generation, inpainting, and anomaly detection at once (Section 5.5). The “one mechanism, many tasks” property is real, but it is a property of the generative orientation, not of PC in general.

The deliverable of this work is therefore a reproducible, honestly-scoped baseline plus three transferable insights: the launch-overhead characterisation of PC inference, the PC-versus-BP confound checklist, and an adversarial-verification protocol that caught a framing error before it reached the page. None of these is a paradigm shift, and none is presented as one. In a subfield whose central empirical question—*when*, if ever, PC does something backpropagation does not—remains open, a clean negative result, a precise systems measurement, and a reusable methodological guard are, we argue, exactly the kind of contribution that moves the discussion forward. Chapter 7 lays out what would be required to extend these claims: greater scale and depth, an Equilibrium-Propagation comparison arm, harder out-of-distribution tests, and the algorithmic and kernel-level work needed to make deep PC both stable and fast. The code, tests, figures, and artifacts are released so that the next study can start where this one ends.

Appendix A

Appendix

A.1 Exact Settling and Weight-Update Derivation

We give the derivation that the implementation verifies against autograd (Chapter 4). With states s_0 (input, clamped to x), $s_1, \dots, s_{n-1}$ (free hidden), and s_n (output, clamped to the one-hot target in training), the top-down prediction and prediction error are

$$\text{pred}_k = W_{k-1} \phi(s_{k-1}) + b_{k-1}, \quad \varepsilon_k = s_k - \text{pred}_k, \quad k = 1, \dots, n,$$

and the energy is $E = \frac{1}{2} \sum_{k=1}^n \|\varepsilon_k\|^2$ (isotropic precision $\Pi = I$).

State gradient (inference). The state s_k enters the energy through ε_k directly ($\partial \varepsilon_k / \partial s_k = I$) and through ε_{k+1} via the prediction $\text{pred}_{k+1} = W_k \phi(s_k) + b_k$, whose Jacobian is $\partial \varepsilon_{k+1} / \partial s_k = -W_k \text{diag}(\phi'(s_k))$. Hence

$$\frac{\partial E}{\partial s_k} = \varepsilon_k - \phi'(s_k) \odot (W_k^\top \varepsilon_{k+1}), \quad \frac{\partial E}{\partial s_n} = \varepsilon_n,$$

and the (Jacobi, all free layers in parallel) settling step is $s_k \leftarrow s_k - \eta_x \partial E / \partial s_k$. The output term drops for the clamped layer; the input term appears only when the input is left free (generation/inpainting, Section 5.5).

Weight gradient (learning). The weight W_i enters only through $\varepsilon_{i+1} = s_{i+1} - W_i \phi(s_i) - b_i$, so $\partial E / \partial W_i = -\varepsilon_{i+1} \phi(s_i)^\top$ and $\partial E / \partial b_i = -\varepsilon_{i+1}$. Gradient *descent* on

E at the settled state gives the purely local Hebbian update

$$W_i \leftarrow W_i + \eta_w \varepsilon_{i+1} \phi(s_i)^\top, \quad b_i \leftarrow b_i + \eta_w \overline{\varepsilon_{i+1}},$$

where $\bar{\cdot}$ is the batch mean. Both gradients are checked against `torch.autograd` to $\sim 10^{-5}$ in the test suite, with autograd used purely as an external oracle and never in the learning path.

A.2 Kernel Implementation Details

The fused settling kernel (Section 4.3; `pcn/kernels/settling_kernel.cu`) runs the whole T -step loop in one launch with the layer states resident in shared memory across steps.

Variants. `v1 (settle_kernel)` assigns one thread block per sample and stages s_k , ε_k , and $\phi(s_k)$ in shared memory; it maximises block parallelism and wins at small batch. `v2 (settle_kernel_tiled)` assigns one block per tile of $TB = 8$ samples, reading each weight element once and reusing it across the tile (the key GEMM optimisation) to win at larger batch; the batch is zero-padded to a multiple of TB . `v3` additionally caches the clamped (hence constant) $\phi(\text{input})$ tile in shared memory, removing $\sim 256 \times$ redundant global reads. `settle()` dispatches `v1/v2` by batch size (`TILED_THRESHOLD=128`).

General depth. A fourth variant (`settle_kernel_deep`) lifts the two-hidden-layer specialisation to arbitrary depth: the per-layer weight, state, and bias buffers are passed as *arrays of device pointers* (encoded as `int64` addresses) together with the per-layer sizes and shared-memory offsets computed at launch, and the same Jacobi update is run over a runtime-variable number of layers. Activations covered are `tanh` (code 0), `identity` (1), and `sigmoid` (2). One portability trap is worth recording: on Windows `long` is 32-bit, so the pointer arrays must be typed `int64_t (long long)`; typing them `long` silently truncates the addresses.

Build and correctness. The `.cu` is compiled at first use via `torch.utils.cpp_extension.load` (no `biber`-style toolchain; needs `nvcc + MSVC`). The correctness gate `scripts/verify_kernel.py` requires `allclose` $\sim 10^{-6}$ against the PyTorch backend across $\{\text{tanh}, \text{identity}, \text{sigmoid}\} \times \text{depths one to four} \times \text{batch } \{32, 256\}$, checked in a *stable* settling regime (an unstable configuration amplifies floating-point reordering into a spurious mismatch).

A.3 Additional Results

All raw artifacts are released under `results/*.json`; the figures (`scripts/make_figures.py`) and tables are generated from them so the plotted and tabulated values share one source of truth. Per-seed values underlying the aggregate statistics are stored in the artifacts — for example the ten per-seed mean-both accuracies of the Song-faithful budget sweep (Section 5.4) in `m4_alternating_song_exact_budgetsweep_s10.json`, the kernel wall-clock and launch counts in `m6_kernel_benchmark.json` / `m6_launch_count.json`, and the depth-ablation accuracies and per-layer signal profiles (Section 5.8) in `signal_decay_w64_T20.json` and `local_decay_fix_verify.json`. The fair PC-vs-BP arms with bootstrap intervals are in `m4_compare3_T40_e8_s3.json`.

A.4 Reproducibility

The project uses `uv` exclusively, with the PyTorch CUDA index pinned to match the local toolkit (cu126). The offline test suite (`uv run pytest -q`, 40 tests) covers the settling and weight-update gradients against autograd, the continual-learning metrics, the API contract, and the EWC/iPC paths, and downloads no data. Each experiment is a script under `scripts/`: `run_experiments.py` (the fair comparison, continual, class-IL, and Song-exact alternating studies), `bench_kernel.py` and `profile_launches.py` (kernel speed and launch count), `verify_kernel.py` (kernel correctness gate), `signal_decay.py` and `local_decay_fix.py` (the depth ablation), and `make_figures.py` (all figures from the JSON artifacts). The optional CUDA kernel is opt-in via the `PCN_CUDA_KERNEL=1` environment flag and a build in an MSVC (`vcvars64`) shell; the default backend is pure PyTorch and fully functional. The complete source, tests, figures, artifacts, and this thesis are public at <https://github.com/AlsoTheZv3n/PCN-self-improvement-research>; the thesis compiles with `tectonic main.tex` (or `latexmk -pdf`).

Bibliography

- Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *NeurIPS*, 2019.
- Rafal Bogacz. A tutorial on the free-energy framework for modelling perception and learning. *Journal of Mathematical Psychology*, 76:198–211, 2017. doi: 10.1016/j.jmp.2015.11.003.
- Greg Diamos, Shubho Sengupta, Bryan Catanzaro, et al. Persistent rnns: Stashing recurrent weights on-chip. *ICML*, 2016.
- Cedric Goemaere, Oliviers, Rafal Bogacz, and Thomas Demeester. Overcoming exponential signal decay in predictive coding (epc). *arXiv preprint arXiv:2505.20137*, 2025.
- Francesco Innocenti, El Mehdi Achour, and Rafal Bogacz. Predictive coding networks in the infinite width and depth limits. *arXiv preprint arXiv:2602.07697*, 2026.
- Francesco Innocenti et al. μ PC: Scaling predictive coding to 100+ layer networks. *arXiv preprint arXiv:2505.13124*, 2025.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.
- Axel Laborieux, Maxence Ernault, Benjamin Scellier, Yoshua Bengio, Julie Grollier, and Damien Querlioz. Scaling equilibrium propagation to deep convnets by drastically reducing its gradient estimator bias. *Frontiers in Neuroscience*, 15:633674, 2021.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

- Vincenzo Lomonaco et al. Avalanche: an end-to-end library for continual learning. *CVPR Workshops; arXiv:2104.00405*, 2021.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. *NeurIPS; arXiv:1706.08840*, 2017.
- Beren Millidge, Anil Seth, and Christopher L. Buckley. Predictive coding: a theoretical and experimental review. *arXiv preprint arXiv:2107.12979*, 2021.
- Beren Millidge, Alexander Tschantz, and Christopher L. Buckley. Predictive coding approximates backprop along arbitrary computation graphs. *Neural Computation*, 34(6):1329–1368, 2022. arXiv:2006.04182.
- Beren Millidge et al. Jpc: Flexible inference for predictive coding networks in jax. *arXiv preprint arXiv:2412.03676*, 2024.
- Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- Luca Pinchetti, Tommaso Salvatori, et al. Benchmarking predictive coding networks (pcx). *arXiv preprint arXiv:2407.01163*, 2024.
- Yuhang Qi, Forasassi, Thomas Lukasiewicz, and Tommaso Salvatori. On the number of inference steps in predictive coding networks. *arXiv preprint arXiv:2506.23800*, 2025.
- Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999. doi: 10.1038/4580.
- Tommaso Salvatori, Yuhang Song, Beren Millidge, et al. A theoretical framework for inference and learning in predictive coding networks (incremental pc). *ICLR; arXiv:2212.00720*, 2024.
- Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.
- Yuhang Song, Beren Millidge, Tommaso Salvatori, Thomas Lukasiewicz, Zhenghua Xu, and Rafal Bogacz. Inferring neural activity before plasticity as a foundation for learning beyond backpropagation. *Nature Neuroscience*, 27(2):348–358, 2024. doi: 10.1038/s41593-023-01514-1.

- van Zwol et al. Preco: a survey and library of predictive coding networks. *arXiv preprint arXiv:2407.04117*, 2024. doi: 10.1145/3797870.
- James C. R. Whittington and Rafal Bogacz. An approximation of the error back-propagation algorithm in a predictive coding network with local hebbian synaptic plasticity. *Neural Computation*, 29(5):1229–1262, 2017. doi: 10.1162/NECO_a_00949.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- Umais Zahid, Qinghai Guo, and Zafeirios Fountas. Predictive coding and curvature. *Neural Computation*, 35(12):1881–1909, 2023. arXiv:2304.02658.